

□ CYBERSÉCURITÉ - Documentation Complète Autosoft

Date de création : 5 Février 2026

Dernière mise à jour : 3 Avril 2026

Version : 1.8.0

□ Table des matières

1. [Vue d'ensemble](#)
2. [Architecture de sécurité](#)
3. [Authentification](#)
4. [Autorisation et contrôle d'accès](#)
5. [Protection des données](#)
6. [Sécurité des API](#)
7. [Middleware de sécurité](#)
8. [Row Level Security \(RLS\)](#)
9. [Gestion des sessions](#)
10. [Cookies sécurisés](#)
11. [VPS ANTS Proxy — Infrastructure sécurisée](#)
12. [Chiffrement des credentials ANTS \(pgcrypto\)](#)
13. [Impersonation sécurisée](#)
14. [Application mobile — Sécurité](#)
15. [Prévention des attaques](#)
16. [Bonnes pratiques implémentées](#)
17. [CI/CD & Qualité du code](#)
18. [Health Check](#)
19. [Audit de sécurité](#)

□ Vue d'ensemble

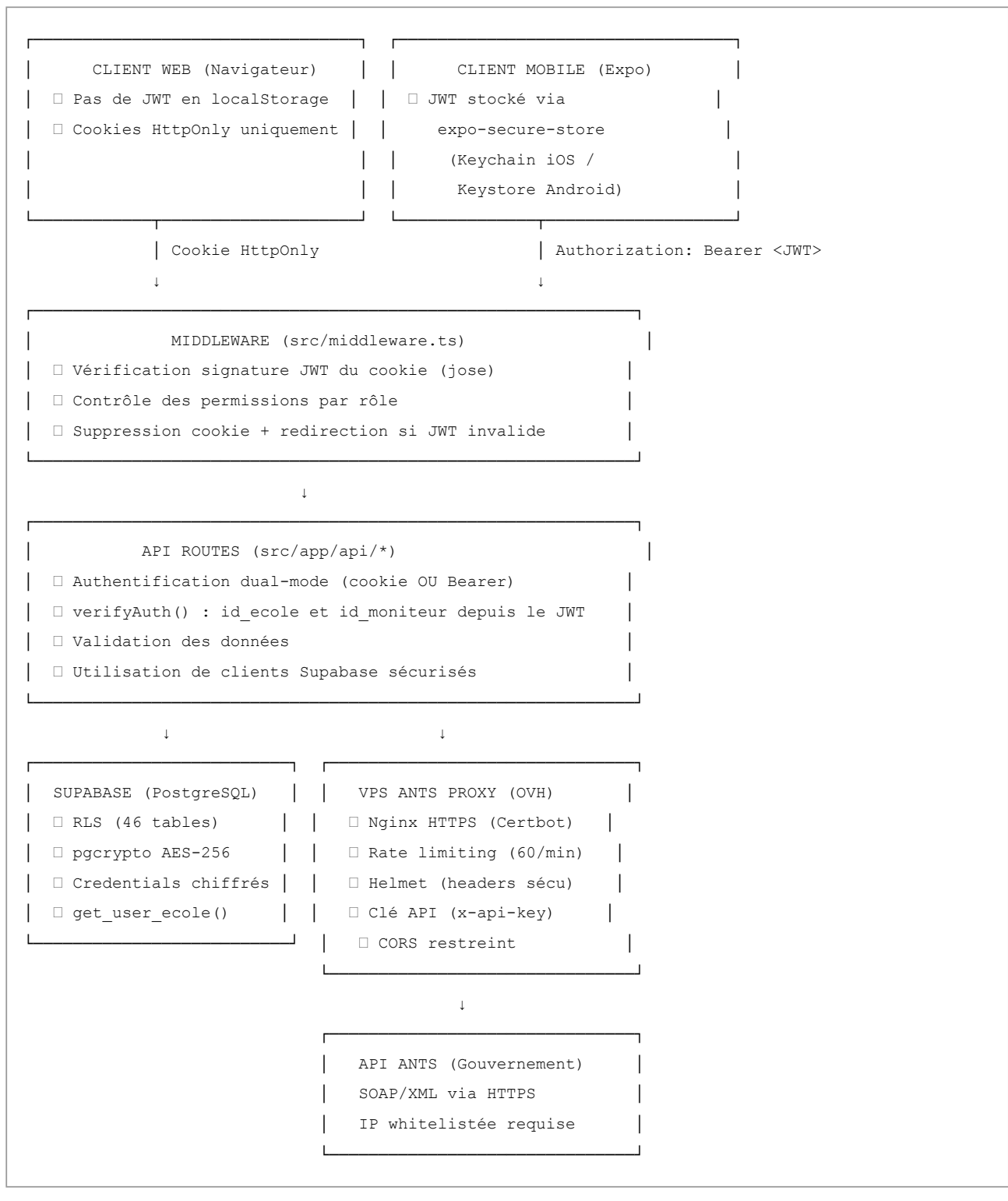
Autosoft implémente une **architecture de sécurité multi-couches** pour protéger les données sensibles des auto-écoles et garantir que chaque utilisateur n'accède qu'aux données auxquelles il a droit.

Principes de sécurité appliqués

- □ **Défense en profondeur** : Plusieurs couches de sécurité (middleware, RLS, validation)
- □ **Principe du moindre privilège** : Chaque utilisateur n'a accès qu'à ce dont il a besoin
- □ **Sécurité par défaut** : Tout est bloqué sauf autorisation explicite
- □ **Validation côté serveur** : Aucune confiance dans les données client
- □ **Cookies HttpOnly** : Protection contre XSS
- □ **Cookie de session signé (JWT)** : Impossible à forger sans le secret serveur

- ☐ **RLS PostgreSQL** : Isolation des données au niveau base de données
 - ☐ **JWT Supabase** : Authentification moderne et sécurisée
 - ☐ **Détection de falsification** : Cookie invalide → suppression + redirection login
 - ☐ **VPS sécurisé (ANTS Proxy)** : Nginx HTTPS, rate limiting, headers de sécurité, clé API
 - ☐ **Chiffrement pgcrypto** : Credentials ANTS chiffrés en AES-256 dans Supabase
 - ☐ **Isolation réseau** : Le VPS est le seul point de contact avec l'API ANTS gouvernementale
 - ☐ **Impersonation sécurisée** : Mécanisme de switch de session pour les admins avec double cookie signé JWT
 - ☐ **Authentification dual-mode** : Cookie HttpOnly (web) + Bearer token (mobile) — un seul `verifyAuth()` pour les deux
 - ☐ **Application mobile sécurisée** : expo-secure-store (Keychain iOS / Keystore Android), Bearer JWT, Error Boundary
 - ☐ **Rate limiting login** : 10 tentatives max / 15 min par IP sur `/api/auth/login` (fenêtre glissante)
 - ☐ **Health check endpoint** : `/api/health` surveille l'API et Supabase en temps réel
 - ☐ **CI/CD pipeline** : GitHub Actions — audit deps + TypeScript + ESLint SAST + tests + build
 - ☐ **ESLint SAST** : `eslint-plugin-security` (détection eval, injection, BiDi) + `react-hooks` + `typescript-eslint`
 - ☐ **Build TypeScript strict** : `ignoreBuildErrors` supprimé — les erreurs TS bloquent le déploiement
 - ☐ **Monitoring Sentry** : capture erreurs serveur/client + tracing performances, RGPD-conforme (`sendDefaultPii: false`)
 - ☐ **Table `audit_log`** : traçabilité des actions sensibles (login réussi/échoué, IP, user-agent), RLS par école, écriture service role uniquement
-

☐ Architecture de sécurité



□ Authentification

1. Système d'authentification

Fichier: src/app/api/auth/login/route.ts

Flux d'authentification

```
1. Utilisateur envoie email + password
  ↓
2. Supabase vérifie les credentials
  ↓
3. Récupération des données utilisateur depuis la table 'utilisateur'
  ↓
4. Signature des données utilisateur dans un JWT (jose, HMAC-SHA256)
  ↓
5. Création de 2 cookies HttpOnly sécurisés :
  - autosoft_session (JWT signé, impossible à forger)
  - sb-access-token (JWT Supabase pour RLS)
  ↓
6. Redirection vers le dashboard approprié selon le rôle
```

Code d'authentification

```

// src/app/api/auth/login/route.ts
import { signSession, SESSION_COOKIE_NAME, SESSION_COOKIE_OPTIONS } from '@lib/security/session';

export async function POST(request: NextRequest) {
  const { email, password } = await request.json();

  // 1. Authentification avec Supabase
  const { data: authData, error: authError } =
    await supabase.auth.signInWithPassword({ email, password });

  if (authError || !authData.user) {
    return NextResponse.json(
      { error: 'Authentification échouée' },
      { status: 401 }
    );
  }

  // 2. Récupération des données utilisateur depuis la BDD
  const { data: userData } = await supabase
    .from('utilisateur')
    .select('*')
    .eq('id_utilisateur', authData.user.id)
    .single();

  // 3. Signer les données dans un JWT (HMAC-SHA256)
  // Le JWT contient les mêmes données qu'avant, mais signées avec SESSION_SECRET
  // → Impossible à forger sans le secret serveur
  const signedToken = await signSession(userDataToStore);
  const sessionCookie = serialize(SESSION_COOKIE_NAME, signedToken, SESSION_COOKIE_OPTIONS);

  // 4. Retour avec les cookies
  const response = NextResponse.json({ success: true, user: userData });
  response.headers.set('Set-Cookie', [sessionCookie, supabaseTokenCookie].join(', '));

  return response;
}

```

2. Sécurité des cookies

Pourquoi HttpOnly ?

❑ SANS HttpOnly (localStorage/sessionStorage)

| Attaque XSS possible |

| → Script malveillant peut lire le token |

| → Vol de session |

❑ AVEC HttpOnly

| Cookie inaccessible en JavaScript |

| → XSS ne peut pas lire le cookie |

| → Session protégée |

Attributs de sécurité des cookies

Attribut	Valeur	Protection
httpOnly	true	❑ Protection XSS - JavaScript ne peut pas lire le cookie
secure	true (prod)	❑ HTTPS uniquement - Pas de transmission en clair
sameSite	lax	❑ Protection CSRF - Limite les requêtes cross-site
maxAge	604800 (7j)	❑ Expiration automatique
path	/	❑ Disponible sur toute l'application
Signature JWT	HS256	❑ Intégrité - Toute modification invalide le cookie

3. Déconnexion sécurisée

Fichier : src/app/api/auth/logout/route.ts

```
export async function POST(request: NextRequest) {
  // 1. Déconnexion Supabase
  const supabase = await createClient();
  await supabase.auth.signOut();

  // 2. Suppression du cookie (maxAge: -1)
  const cookie = serialize('autosoft_session', '', {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge: -1, // ❑ Expire immédiatement
    path: '/'
  });

  const response = NextResponse.json({ success: true });
  response.headers.set('Set-Cookie', cookie);

  return response;
}
```

□ Autorisation et contrôle d'accès

1. Configuration des permissions

Fichier: src/lib/security/auth-config.ts

Rôles utilisateurs

```
export type UserRole =  
  | 'admin'          // Super-admin, accès total  
  | 'directeur'      // Directeur d'auto-école  
  | 'moniteur'       // Moniteur de conduite  
  | 'secretaire'     // Secrétaire  
  | 'comptable'      // Comptable  
  | 'autosoft'       // Super-admin Autosoft  
  | 'eleves';        // Élève
```

Mapping des routes et permissions

```
export const ROUTE_PERMISSIONS: Record<string, UserRole[]> = {  
  '/autosoft': ['autosoft', 'admin'],  
  '/admin':    ['admin', 'autosoft'],  
  '/directeur': ['directeur', 'admin', 'autosoft'],  
  '/moniteur': ['moniteur', 'directeur', 'admin', 'autosoft'],  
  '/secretaire': ['secretaire', 'directeur', 'admin', 'autosoft'],  
  '/comptable': ['comptable', 'directeur', 'admin', 'autosoft'],  
  '/eleves':   ['eleves', 'directeur', 'admin', 'autosoft'],  
};
```

Routes publiques

```
export const PUBLIC_ROUTES = [  
  '/login',  
  '/register',  
  '/forgot-password',  
  '/reset-password',  
  '/verify-email',  
  '/',  
];  
  
export const PUBLIC_API_ROUTES = [  
  '/api/auth',  
  '/api/public',  
];
```

2. Helpers de vérification

Fichier: src/lib/security/auth-helpers.ts

Vérification d'authentification

```
// src/lib/security/auth-helpers.ts
import { verifySession, SESSION_COOKIE_NAME } from '@lib/security/session';

export async function verifyAuth(request: NextRequest): Promise<AuthResult> {
  let token: string | undefined;

  // 1. Mode dual : essayer d'abord le Bearer token (mobile),
  //    puis le cookie HttpOnly (web)
  const authHeader = request.headers.get('authorization');
  if (authHeader?.startsWith('Bearer ')) {
    token = authHeader.slice(7);
  } else {
    const cookieHeader = request.headers.get('cookie');
    if (cookieHeader) {
      const cookies = parse(cookieHeader);
      token = cookies[SESSION_COOKIE_NAME];
    }
  }

  if (!token) {
    return { success: false, error: 'Non authentifié - Aucun token' };
  }

  // 2. Vérifier la signature JWT et décoder les données
  // Le même JWT signé HS256 est utilisé pour les deux modes
  const userData = await verifySession(token);

  if (!userData) {
    return { success: false, error: 'Session invalide - JWT invalide ou expiré' };
  }

  return { success: true, user: userData as AuthenticatedUser };
}
```

Vérification des permissions

```
export function checkPermissions(
  user: AuthenticatedUser,
  allowedRoles: AuthenticatedUser['role'][]
): boolean {
  return allowedRoles.includes(user.role);
}
```

Vérification d'accès à une école


```
export function checkEcoleAccess(  
  user: AuthenticatedUser,  
  id_ecole: number  
): boolean {  
  // Les admins peuvent accéder à toutes les écoles  
  if (user.role === 'admin') {  
    return true;  
  }  
  
  // Sinon, vérifier que l'utilisateur appartient à cette école  
  return user.id_ecole === id_ecole;  
}
```

Vérification d'accès à un bureau

```
export function checkBureauAccess(  
  user: AuthenticatedUser,  
  id_bureau: number  
): boolean {  
  // Les admins et directeurs peuvent accéder à tous les bureaux  
  if (user.role === 'admin' || user.role === 'directeur') {  
    return true;  
  }  
  
  // id_bureau = 0 signifie "tous les bureaux"  
  if (id_bureau === 0) {  
    return false; // Uniquement admins/directeurs  
  }  
  
  return user.id_bureau === id_bureau;  
}
```

☐ Middleware de sécurité

Fichier: src/middleware.ts

Rôle du middleware

Le middleware s'exécute **côté serveur** sur **CHAQUE requête** avant qu'elle n'atteigne les pages ou les API routes.

Flux de vérification

```
1. Requête entrante
  ↓
2. Ignorer ressources statiques (_next, images, fonts)
  ↓
3. Autoriser routes publiques (/login, /register)
  ↓
4. Vérifier présence du cookie HttpOnly
  ↓
5. Vérifier la signature JWT du cookie (jose, HS256)
  → Si invalide : supprimer cookie + redirection /login
  ↓
6. Vérifier les permissions pour la route demandée
  ↓
7. Autoriser ou rediriger
```

Code du middleware

```

import { verifySession, SESSION_COOKIE_NAME } from '@lib/security/session';

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // 1. Ignorer les ressources statiques
  if (pathname.startsWith('/_next') || pathname === '/favicon.ico') {
    return NextResponse.next();
  }

  // 2. Autoriser les routes publiques
  if (PUBLIC_ROUTES.includes(pathname)) {
    return NextResponse.next();
  }

  // 3. Vérifier le cookie HttpOnly
  const cookieHeader = request.headers.get('cookie');
  if (!cookieHeader) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // 4. Parser le cookie
  const cookies = parse(cookieHeader);
  const sessionCookie = cookies[SESSION_COOKIE_NAME];
  if (!sessionCookie) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // 5. Vérifier la signature JWT (jose, HS256)
  // Si le cookie a été modifié → signature invalide → user = null
  const user = await verifySession(sessionCookie);

  if (!user) {
    // □ Supprimer le cookie forgé/corrompu + redirection
    const loginUrl = new URL('/login', request.url);
    loginUrl.searchParams.set('reason', 'invalid_session');
    const response = NextResponse.redirect(loginUrl);
    response.cookies.set(SESSION_COOKIE_NAME, '', { maxAge: -1, path: '/' });
    return response;
  }

  // 6. Vérifier les permissions
  for (const [routePrefix, allowedRoles] of Object.entries(ROUTE_PERMISSIONS)) {
    if (pathname.startsWith(routePrefix)) {
      if (!allowedRoles.includes(user.role)) {
        const dashboardUrl = new URL(ROLE_DASHBOARDS[user.role] || '/login', request.url);
        return NextResponse.redirect(dashboardUrl);
      }
    }
  }
  break;
}

```

```
    }  
  }  
  
  // 7. Accès autorisé  
  return NextResponse.next();  
}
```

Configuration du matcher

```
export const config = {  
  matcher: [  
    // Match toutes les routes sauf :  
    // - /_next (Next.js internals)  
    // - /fonts, /images (ressources statiques)  
    // - /favicon.ico, /robots.txt  
    '(!(_next|fonts|images|favicon.ico|robots.txt).*)',  
  ],  
};
```

□ Row Level Security (RLS)

Qu'est-ce que RLS ?

Row Level Security est une fonctionnalité PostgreSQL qui filtre automatiquement les données **au niveau de la base de données** en fonction de l'utilisateur connecté.

Avantages de RLS

- **Sécurité au niveau BDD** : Même si un hacker contourne l'application, RLS protège les données
- **Impossible à bypasser** : Les politiques sont appliquées par PostgreSQL
- **Automatique** : Pas besoin d'ajouter `WHERE id_ecole = x` partout
- **Centralisé** : Toutes les règles au même endroit

Fonctions de sécurité

Fichier : `database/migration_enable_rls.sql`

1. Récupérer l'école de l'utilisateur

```
CREATE OR REPLACE FUNCTION get_user_ecole()
RETURNS INTEGER AS $$
BEGIN
    RETURN (
        SELECT id_ecole
        FROM public.utilisateur
        WHERE id_utilisateur = auth.uid()::text
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

2. Récupérer le bureau de l'utilisateur

```
CREATE OR REPLACE FUNCTION get_user_bureau()
RETURNS INTEGER AS $$
BEGIN
    RETURN (
        SELECT id_bureau
        FROM public.utilisateur
        WHERE id_utilisateur = auth.uid()::text
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

3. Récupérer le rôle de l'utilisateur

```
CREATE OR REPLACE FUNCTION get_user_role()
RETURNS TEXT AS $$
BEGIN
    RETURN (
        SELECT role
        FROM public.utilisateur
        WHERE id_utilisateur = auth.uid()::text
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Politiques RLS

Exemple 1 : Table `bureau` (isolation par école uniquement)

```
-- Activer RLS
ALTER TABLE public.bureau ENABLE ROW LEVEL SECURITY;

-- Politique : Voir uniquement les bureaux de son école
DROP POLICY IF EXISTS "Isolation par école" ON public.bureau;
CREATE POLICY "Isolation par école"
  ON public.bureau FOR ALL
  USING (id_ecole = get_user_ecole());
```

Exemple 2 : Table eleves (isolation par école ET bureau)

```
-- Activer RLS
ALTER TABLE public.eleves ENABLE ROW LEVEL SECURITY;

-- Politique : Voir uniquement les élèves de son école et bureau
-- SAUF si id_bureau = 0 (bureau virtuel "Tout")
DROP POLICY IF EXISTS "Isolation par école et bureau" ON public.eleves;
CREATE POLICY "Isolation par école et bureau"
  ON public.eleves FOR ALL
  USING (
    id_ecole = get_user_ecole()
    AND (
      get_user_bureau() = 0          -- Bureau "Tout"
      OR get_user_bureau() IS NULL   -- Pas de bureau
      OR id_bureau = get_user_bureau() -- Bureau spécifique
    )
  );
```

Gestion du bureau virtuel "Tout"

Certains utilisateurs (directeurs, admins) ont `id_bureau = 0` pour accéder à **tous les bureaux** de leur école.

```
-- Politique qui gère le bureau "Tout"
USING (
  id_ecole = get_user_ecole()
  AND (
    get_user_bureau() = 0          -- ☐ Si bureau = 0, voir TOUT
    OR get_user_bureau() IS NULL   -- ☐ Si NULL, voir TOUT
    OR id_bureau = get_user_bureau() -- ☐ Sinon, bureau spécifique
  )
)
```

Tables protégées par RLS

46 tables ont RLS activé avec isolation par école et bureau :

- ants_credentials
- ants_demandes

- ants_pieces_jointes
- auto_ecole
- bilan
- bureau
- certifications
- competences_eleve
- contrats
- contrats_eleves
- demandes_contact
- depense
- depense_recurrente
- depense_vehicule
- devis
- disponibilite
- documents
- echeances_paiement
- echelonnements
- eleves
- enseignants
- entretien_vehicule
- evaluations_sous_competences
- examen_resultats
- facture
- forfait
- forfait_prestations
- formation
- kilometrage_vehicule
- manoeuvres
- notation
- notifications
- observations_eleves
- paiements
- payeurs
- permis
- planning
- prestations
- prestations_contrats
- progressions_sous_competences
- promotions
- question_reponses
- remboursements
- remises_contrats
- representant_legal
- reservation_vehicule
- signatures_contrats
- statistique
- suivi_conduite_accompagnee
- transactions

- utilisateur
- utilisations_promotions
- vehicule

2 tables de référence exemptées de RLS (données partagées) : type_permis, sous_competences

□ Gestion des sessions

1. Session signée avec JWT (jose)

Fichier : src/lib/security/session.ts

```
import { SignJWT, jwtVerify } from 'jose';

const SECRET = new TextEncoder().encode(process.env.SESSION_SECRET);

// Signer les données utilisateur dans un JWT
export async function signSession(data: SessionData): Promise<string> {
  return await new SignJWT(data)
    .setProtectedHeader({ alg: 'HS256' }) // HMAC-SHA256
    .setIssuedAt() // Date de création
    .setExpirationTime('7d') // Expire dans 7 jours
    .sign(SECRET); // Signe avec le secret
}

// Vérifier et décoder un JWT signé
// Retourne null si la signature est invalide ou le JWT expiré
export async function verifySession(token: string): Promise<SessionData | null> {
  try {
    const { payload } = await jwtVerify(token, SECRET);
    if (!payload.id_utilisateur || !payload.email || !payload.role) return null;
    return payload as unknown as SessionData;
  } catch {
    return null; // Signature invalide, expiré, ou format incorrect
  }
}
```

2. Détection de falsification côté client

Fichier : src/contexts/UserContext.tsx

Si /api/auth/me retourne 401 (cookie invalide/forgé), le client redirige automatiquement vers /login :


```

if (response.status === 401) {
  setUser({});
  if (window.location.pathname !== '/login') {
    window.location.href = '/login?reason=invalid_session';
  }
}

```

3. Client Supabase avec RLS

Fichier: src/lib/supabase-rls.ts

```

import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';

export async function createClientWithRLS() {
  const cookieStore = await cookies();

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Peut être ignoré si middleware rafraîchit les sessions
          }
        },
      },
    }
  );

  return supabase;
}

```

2. Auto-refresh des tokens

Le client `@supabase/ssr` **rafraîchit automatiquement** les tokens expirés :

Token expire dans < 5 minutes
↓
Auto-refresh par @supabase/ssr
↓
Nouveau token généré
↓
Cookie mis à jour automatiquement

3. Vérification de session

Fichier : src/lib/security/verify-auth.ts

```
export async function verifySession(supabase: SupabaseClient): Promise<AuthUser | null> {  
  // 1. Récupérer la session depuis les cookies  
  const { data: { user }, error } = await supabase.auth.getUser();  
  
  if (error || !user) {  
    return null;  
  }  
  
  // 2. Récupérer le VRAI rôle depuis la BDD (pas depuis le client !)  
  const { data: userData } = await supabase  
    .from('utilisateur')  
    .select('id_utilisateur, email, role, id_ecole, id_bureau')  
    .eq('id_utilisateur', user.id)  
    .single();  
  
  if (!userData) {  
    return null;  
  }  
  
  return {  
    id: userData.id_utilisateur,  
    email: userData.email,  
    role: userData.role,  
    id_ecole: userData.id_ecole,  
    id_bureau: userData.id_bureau,  
  };  
}
```

□ Sécurité des API

1. Validation des données

Toutes les API routes valident les données entrantes :

```
export async function POST(request: NextRequest) {
  const body = await request.json();

  // Validation des champs obligatoires
  if (!body.nom || !body.email) {
    return NextResponse.json(
      { success: false, error: 'Champs obligatoires manquants' },
      { status: 400 }
    );
  }

  // Validation du format email
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(body.email)) {
    return NextResponse.json(
      { success: false, error: 'Email invalide' },
      { status: 400 }
    );
  }

  // Suite du traitement...
}
```

2. Vérification des permissions dans les API

Mise à jour v1.5.0 : Toutes les routes API (105+) sont désormais protégées par `verifyAuth()`. L'`id_ecole` et l'`id_moniteur` sont **toujours lus depuis le JWT** (non falsifiable), jamais depuis le `body` ou les `query params`.

```
import { verifyAuth, checkEcoleAccess } from '@lib/security/auth-helpers';

export async function GET(request: NextRequest) {
  // 1. Vérifier l'authentification (dual-mode : cookie OU Bearer)
  const authResult = await verifyAuth(request);

  if (!authResult.success || !authResult.user) {
    return NextResponse.json(
      { success: false, error: authResult.error },
      { status: 401 }
    );
  }

  // 2. □ id_ecole depuis le JWT (pas depuis les query params !)
  const id_ecole = authResult.user.id_ecole;

  // 3. Utiliser le client Supabase filtré par RLS
  const supabase = await createClientWithRLS();
  const { data } = await supabase
    .from('eleves')
    .select('*')
    .eq('id_ecole', id_ecole);

  // 4. Continuer le traitement...
}
```

3. Utilisation des bons clients Supabase

Client	Usage	Fichier
<code>createClientWithRLS()</code>	Routes API avec RLS	<code>src/lib/supabase-rls.ts</code>
<code>createClient()</code>	Routes auth (login, logout)	<code>src/lib/supabase-server.ts</code>
<code>supabaseAdmin</code>	Opérations admin (bypass RLS)	<code>src/lib/supabase-admin.ts</code>

```
// ❑ BON : Utiliser createClientWithRLS pour les données utilisateur
import { createClientWithRLS } from '@lib/supabase-rls';

export async function GET(request: NextRequest) {
  const supabase = await createClientWithRLS();

  // RLS appliqué automatiquement
  const { data } = await supabase
    .from('eleves')
    .select('*');

  return NextResponse.json({ eleves: data });
}

// ❑ MAUVAIS : Utiliser supabaseAdmin sans vérification
import { supabaseAdmin } from '@lib/supabase-admin';

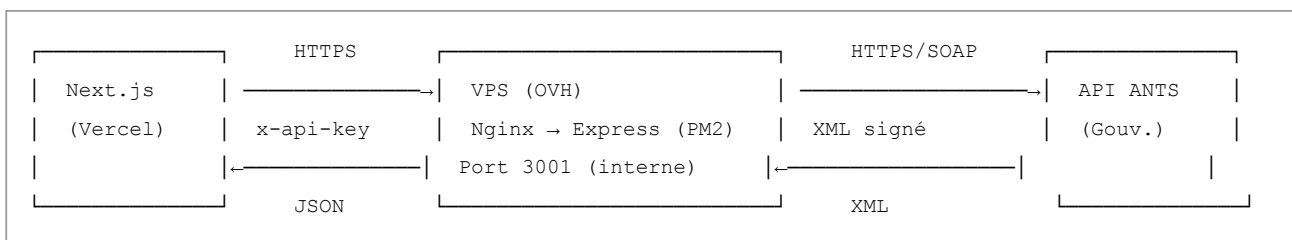
export async function GET(request: NextRequest) {
  // ⚠ DANGER : Bypass RLS, retourne TOUTES les données !
  const { data } = await supabaseAdmin
    .from('eleves')
    .select('*');

  return NextResponse.json({ eleves: data });
}
```

❓ VPS ANTS Proxy — Infrastructure sécurisée

1. Architecture réseau

Le VPS ANTS Proxy est un serveur dédié OVH qui sert d'**unique point de contact** entre Autosoft et l'API ANTS gouvernementale (SOAP/XML). Cette architecture isole complètement l'API ANTS du frontend.



Pourquoi un VPS dédié ?

- ❑ **IP fixe whitelisted** : L'ANTS n'autorise que certaines IP à accéder à leur API
- ❑ **Isolation** : Les appels SOAP ne transitent jamais par le navigateur client
- ❑ **Contrôle total** : Firewall, rate limiting, logs d'accès côté serveur
- ❑ **Sécurité des credentials** : Les identifiants ANTS ne sont jamais exposés côté client

2. Nginx — Reverse proxy HTTPS

Fichier : /etc/nginx/sites-available/ants-proxy

Le serveur Nginx agit comme reverse proxy devant Express et fournit :

Certificat SSL/TLS (Let's Encrypt)

```
server {  
    listen 443 ssl;  
    server_name vps-lac08b61.vps.ovh.net;  
  
    ssl_certificate /etc/letsencrypt/live/vps-lac08b61.vps.ovh.net/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/vps-lac08b61.vps.ovh.net/privkey.pem;  
    include /etc/letsencrypt/options-ssl-nginx.conf;  
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;  
}
```

- ☐ **HTTPS obligatoire** : Tout le trafic est chiffré TLS
- ☐ **Certbot** : Renouvellement automatique des certificats Let's Encrypt
- ☐ **Redirection HTTP → HTTPS** : Le port 80 redirige vers 443

Headers de sécurité

```
# Headers de sécurité  
add_header X-Frame-Options "DENY" always;  
add_header X-Content-Type-Options "nosniff" always;  
add_header X-XSS-Protection "1; mode=block" always;  
add_header Referrer-Policy "strict-origin-when-cross-origin" always;  
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

Header

Protection

X-Frame-Options: DENY	Empêche l'intégration en iframe (clickjacking)
X-Content-Type-Options: nosniff	Empêche le MIME sniffing
X-XSS-Protection: 1; mode=block	Protection XSS navigateur
Referrer-Policy	Limite les informations de referer
Strict-Transport-Security	Force HTTPS pendant 1 an (HSTS)

Rate limiting Nginx

```
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;  
  
location / {  
    limit_req zone=api burst=20 nodelay;  
    proxy_pass http://127.0.0.1:3001;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}
```

- ☐ **10 requêtes/seconde** par IP au niveau Nginx
- ☐ **Burst de 20** : Absorbe les pics légitimes

- ☐ **X-Forwarded-For** : Transmet l'IP réelle à Express

3. Express — Sécurité applicative

Fichier : `vps-ants-proxy/src/index.ts`

Trust proxy

```
// Nécessaire car derrière Nginx
app.set('trust proxy', 1);
```

Express fait confiance au premier proxy (Nginx) pour identifier l'IP réelle des clients via `X-Forwarded-For`.

Helmet (headers HTTP)

```
app.use(helmet());
```

Helmet ajoute automatiquement des headers de sécurité HTTP supplémentaires (CSP, HSTS, etc.).

CORS restreint

```
const ALLOWED_ORIGINS = [
  'https://www.auto-soft.fr',
  'https://auto-soft.fr',
  process.env.CORS_EXTRA_ORIGIN, // optionnel pour dev/preview
].filter(Boolean) as string[];

app.use(cors({
  origin: (origin, callback) => {
    if (!origin) return callback(null, true); // health checks
    if (ALLOWED_ORIGINS.some(allowed => origin === allowed || origin.endsWith('.vercel.app'))) {
      return callback(null, true);
    }
    callback(new Error('CORS non autorisé'));
  },
  methods: ['GET', 'POST'],
  allowedHeaders: ['Content-Type', 'x-api-key'],
}));
```

- ☐ **Domaines autorisés** : Uniquement `auto-soft.fr` et les déploiements Vercel
- ☐ **Méthodes limitées** : GET et POST uniquement
- ☐ **Headers limités** : Seuls `Content-Type` et `x-api-key` sont acceptés

Rate limiting Express

```

app.use('/ants', rateLimit({
  windowMs: 60 * 1000,          // Fenêtre de 1 minute
  max: 60,                      // Max 60 requêtes par IP
  standardHeaders: true,
  legacyHeaders: false,
  message: { success: false, error: 'Trop de requêtes, réessayez dans 1 minute' },
}));

```

Double rate limiting : Nginx (10/s) + Express (60/min) pour une protection en profondeur.

Authentification par clé API

```

// Refuser de démarrer sans clé API valide
if (!API_KEY || API_KEY.length < 32) {
  console.error('❌ FATAL: API_KEY manquante ou trop courte');
  process.exit(1);
}

// Middleware de vérification
function authMiddleware(req, res, next) {
  const apiKey = req.headers['x-api-key'];
  if (!apiKey || apiKey !== API_KEY) {
    res.status(401).json({ success: false, error: 'Clé API invalide' });
    return;
  }
  next();
}

app.use('/ants', authMiddleware);

```

- ☐ **Clé API 256 bits** : Générée avec `openssl rand -hex 32`
- ☐ **Vérification sur chaque requête** `/ants/*`
- ☐ **Refus de démarrer** si la clé est absente ou trop courte (< 32 caractères)

Protection XML Injection


```
function escapeXml(str: string): string {
    return str
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&apos;');
}

// Validation base64 pour les pièces jointes
if (!/^[A-Za-z0-9+/=\\s]+$/.test(fichier)) {
    res.status(400).json({ success: false, error: 'base64 invalide' });
    return;
}
```

- ☐ **Échappement XML** : Toutes les données utilisateur sont échappées avant injection dans le SOAP
- ☐ **Validation base64** : Les fichiers PJ sont validés comme base64 pur (pas d'injection XML)

Messages d'erreur sanitisés

```
function safeErrorMessage(error: unknown): string {
    if (error instanceof Error) {
        if (error.name === 'AbortError') return 'Timeout: l\'ANTS n\'a pas répondu';
        if (error.message.startsWith('ANTS HTTP')) return error.message;
        return 'Erreur de communication avec l\'ANTS'; // Message générique
    }
    return 'Erreur inconnue';
}
```

- ☐ **Pas de fuite d'informations** : Les erreurs internes ne sont jamais exposées au client
- ☐ **Timeout** : 30 secondes max par appel SOAP (`AbortController`)

4. PM2 — Gestion de processus

```
pm2 start dist/index.js --name autosoft-ants-proxy
pm2 startup      # Auto-démarrage au boot
pm2 save         # Sauvegarder la config
```

- ☐ **Redémarrage automatique** : En cas de crash, PM2 relance le processus
- ☐ **Auto-startup** : Le proxy redémarre automatiquement après un reboot du VPS
- ☐ **Logs centralisés** : `pm2 logs autosoft-ants-proxy`

5. Certificat CA ANTS

Le serveur ANTS utilise un certificat signé par une autorité de certification gouvernementale (AC Services Applicatifs SA SN5).

Node.js ne la reconnaît pas nativement.

- **Fichier** : `/opt/autosoft/certs/ac_services_applicatifs_qlf_5.pem` (converti de DER en PEM)
- **Variable** : `NODE_EXTRA_CA_CERTS` configurée dans `.env` et transmise via PM2

- ☐ Sans ce certificat, toute connexion TLS vers l'ANTS échoue

Documentation complète : Voir `VPS-ANTS-PROXY.md` pour l'architecture détaillée, les commandes SSH, la gestion multi-écoles, et le guide de déploiement.

6. Variables d'environnement VPS

Variable	Description	Sécurité
<code>API_KEY</code>	Clé API 256 bits pour authentifier les requêtes	☐ Jamais exposée côté client
<code>ANTS_SOAP_URL</code>	URL endpoint SOAP ANTS (qualification ou production)	☐ Configurable par environnement
<code>ANTS_SOAP_ACTION_BASE</code>	Base URL pour les headers SOAPAction	☐ Côté serveur uniquement
<code>ANTS_NAMESPACE</code>	Namespace XML SOAP (identifiant, pas une URL serveur)	☐ Côté serveur uniquement
<code>ANTS_CODE_EDITEUR</code>	Code éditeur Autosoft auprès de l'ANTS (1671)	☐ Côté serveur uniquement
<code>NODE_EXTRA_CA_CERTS</code>	Chemin vers le certificat CA ANTS (PEM)	☐ Côté serveur uniquement
<code>PORT</code>	Port interne Express (3001)	☐ Non exposé (Nginx proxy)
<code>CORS_EXTRA_ORIGIN</code>	Origine CORS supplémentaire (dev)	☐ Optionnel

7. Variables d'environnement Next.js (côté Vercel)

Variable	Description	Sécurité
<code>ANTS_VPS_URL</code>	URL HTTPS du VPS (<code>https://vps-xxx.vps.ovh.net</code>)	☐ Côté serveur uniquement
<code>ANTS_VPS_API_KEY</code>	Même clé que <code>API_KEY</code> du VPS	☐ Jamais exposée côté client
<code>ANTS_ENCRYPTION_KEY</code>	Clé AES-256 pour pgcrypto	☐ Côté serveur uniquement

☐ Chiffrement des credentials ANTS (pgcrypto)

1. Pourquoi chiffrer ?

Les auto-écoles saisissent leurs identifiants ANTS (identifiant, mot de passe, numéro d'agrément) dans l'application. Ces données **extrêmement sensibles** sont stockées chiffrées dans Supabase.

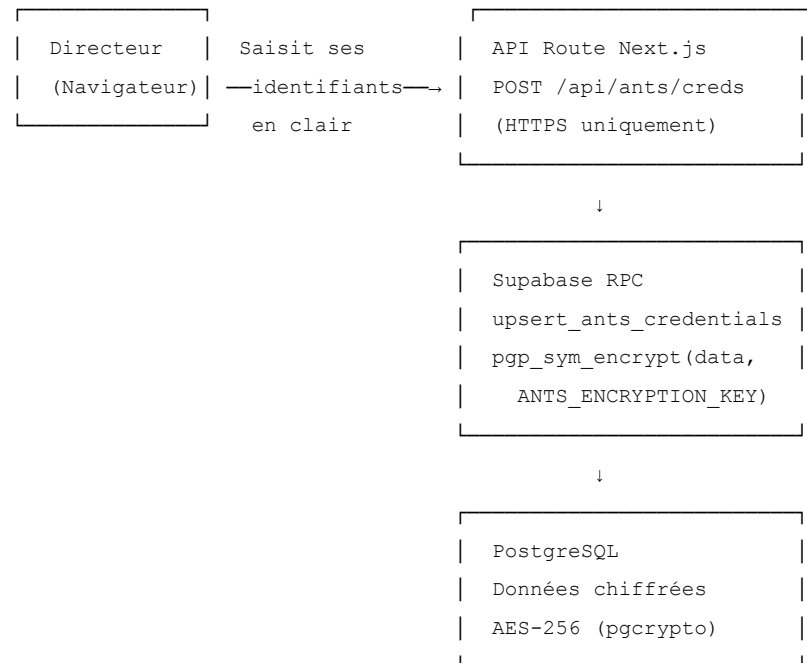
❑ SANS chiffrement

Table ants_credentials	
identifiant: "jean.dupont"	
mot_de_passe: "MonMotDePasse123"	← Lisible en clair !
numero_agrement: "A1234"	

❑ AVEC pgcrypto (AES-256)

Table ants_credentials	
identifiant: \xc30d04070302...	
mot_de_passe: \xc30d04070302...	← Chiffré AES-256
numero_agrement: \xc30d04070302...	

2. Architecture de chiffrement



3. Fonctions RPC Supabase

Chiffrement (upsert)

```

-- Fonction RPC : upsert_ants_credentials
-- Chiffre les credentials avec pgp_sym_encrypt avant stockage
CREATE OR REPLACE FUNCTION upsert_ants_credentials(
    p_id_ecole INTEGER,
    p_identifiant TEXT,
    p_mot_de_passe TEXT,
    p_numero_agrement TEXT,
    p_encryption_key TEXT
) RETURNS VOID AS $$
BEGIN
    INSERT INTO ants_credentials (id_ecole, ants_identifiant, ants_mot_de_passe, ants_numero_agrement)
    VALUES (
        p_id_ecole,
        pgp_sym_encrypt(p_identifiant, p_encryption_key),
        pgp_sym_encrypt(p_mot_de_passe, p_encryption_key),
        pgp_sym_encrypt(p_numero_agrement, p_encryption_key)
    )
    ON CONFLICT (id_ecole) DO UPDATE SET
        ants_identifiant = pgp_sym_encrypt(p_identifiant, p_encryption_key),
        ants_mot_de_passe = pgp_sym_encrypt(p_mot_de_passe, p_encryption_key),
        ants_numero_agrement = pgp_sym_encrypt(p_numero_agrement, p_encryption_key),
        updated_at = NOW();
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Déchiffrement (lecture)

```

-- Fonction RPC : get_ants_credentials
-- Déchiffre les credentials avec pgp_sym_decrypt
CREATE OR REPLACE FUNCTION get_ants_credentials(
    p_id_ecole INTEGER,
    p_encryption_key TEXT
) RETURNS TABLE (
    ants_identifiant TEXT,
    ants_mot_de_passe TEXT,
    ants_numero_agrement TEXT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        pgp_sym_decrypt(c.ants_identifiant, p_encryption_key),
        pgp_sym_decrypt(c.ants_mot_de_passe, p_encryption_key),
        pgp_sym_decrypt(c.ants_numero_agrement, p_encryption_key)
    FROM ants_credentials c
    WHERE c.id_ecole = p_id_ecole;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

4. Flux de déchiffrement pour les appels ANTS

```
1. API Route Next.js reçoit une demande ANTS
  ↓
2. getAntsCredentials(id_ecole) appelle Supabase RPC
   → pgp_sym_decrypt avec ANTS_ENCRYPTION_KEY
  ↓
3. Credentials déchiffrés en mémoire (jamais stockés en clair)
  ↓
4. Envoi au VPS proxy via HTTPS + x-api-key
  ↓
5. VPS construit le SOAP XML et appelle l'ANTS
  ↓
6. Credentials effacés de la mémoire après l'appel
```

5. Sécurité du mot de passe dans l'UI

```
// GET /api/ants/credentials - Ne retourne JAMAIS le mot de passe en clair
return NextResponse.json({
  success: true,
  configured: true,
  credentials: {
    identifiant: row.ants_identifiant,
    motDePasse: '.....', // ← Masqué côté serveur
    numeroAgrement: row.ants_numero_agrement,
  },
});
```

- ☐ **Mot de passe masqué** : L'API GET ne retourne jamais le mot de passe en clair
- ☐ **Déchiffrement uniquement pour les appels ANTS** : Le mot de passe n'est déchiffré que pour construire la requête SOAP
- ☐ **SECURITY DEFINER** : Les fonctions RPC s'exécutent avec les privilèges du créateur, pas de l'appelant
- ☐ **Clé de chiffrement côté serveur** : ANTS_ENCRYPTION_KEY n'est jamais exposée au client

☐ Impersonation sécurisée

1. Objectif

Permettre aux utilisateurs **admin** et **autosoft** de se connecter temporairement en tant que directeur d'une auto-école spécifique, pour diagnostiquer des problèmes ou assister un client, **sans connaître ses identifiants**.

2. Architecture



3. Flux détaillé

Activation de l'impersonation

```
// POST /api/auth/impersonate - src/app/api/auth/impersonate/route.ts
```

1. Vérifier la session JWT (autosoft_session)
↓
2. Si impersonation déjà active → vérifier le rôle du cookie ORIGINAL
Sinon → vérifier le rôle courant
↓
3. Seuls les rôles ['admin', 'autosoft'] sont autorisés
↓
4. Valider id_ecole : entier positif + existe en BDD (via supabaseAdmin)
↓
5. Sauvegarder la session originale dans `autosoft\_original\_session`
(seulement si première impersonation, pas d'écrasement)
↓
6. Signer une nouvelle session :

```
{  
  id_utilisateur: admin.id, // ← Garde l'ID admin pour traçabilité  
  email: admin.email,  
  role: 'directeur',        // ← Nouveau rôle  
  id_ecole: 42,             // ← École cible  
  id_bureau: 0,            // ← Accès tous bureaux  
}
```


↓
7. Poser les 2 cookies HttpOnly signés JWT
↓
8. Log serveur : "[Impersonate] □ admin@autosoft.fr (admin) → directeur de 'Mon Auto-École'"

Sortie de l'impersonation

```
// POST /api/auth/impersonate/exit - src/app/api/auth/impersonate/exit/route.ts
```

1. Récupérer le cookie `autosoft\_original\_session`
↓
2. Vérifier que le JWT original est encore valide (non expiré)
→ Si expiré : supprimer les 2 cookies + forcer re-login
↓
3. Restaurer le JWT original dans `autosoft\_session`
↓
4. Supprimer le cookie `autosoft\_original\_session` (maxAge: -1)
↓
5. Rediriger vers /autosoft/dashboard ou /admin/dashboard

Déconnexion pendant l'impersonation

```
// POST /api/auth/logout - src/app/api/auth/logout/route.ts

// ☐ Supprime les DEUX cookies pour éviter les fuites
const deleteSession = serialize('autosoft_session', '', { maxAge: -1 });
const deleteOriginalSession = serialize('autosoft_original_session', '', { maxAge: -1 });
response.headers.set('Set-Cookie', [deleteSession, deleteOriginalSession].join(', '));
```

4. Bypass RLS en mode impersonation

Fichier: src/lib/supabase-rls.ts

En mode impersonation, le JWT Supabase de l'admin ne correspond pas à l'école cible → le RLS bloquerait toutes les requêtes. Solution :

```
export async function createClientWithRLS() {
  const cookieStore = await cookies();

  // Vérifier si une impersonation est active
  const originalSessionCookie = cookieStore.get('autosoft_original_session')?.value;
  if (originalSessionCookie) {
    const originalUser = await verifySession(originalSessionCookie);
    if (originalUser && ['admin', 'autosoft'].includes(originalUser.role)) {
      // ☐ Impersonation légitime → bypass RLS via supabaseAdmin
      return supabaseAdmin;
    }
  }

  // Mode normal → client avec RLS
  return createServerClient(/* ... */);
}
```

Pourquoi c'est sécurisé :

- ☐ Le cookie `autosoft_original_session` est **HttpOnly** → impossible à poser via JavaScript
- ☐ Le cookie est un **JWT signé** (HS256) → impossible à forger sans `SESSION_SECRET`
- ☐ Le rôle est vérifié dans le JWT → seuls `admin/autosoft` déclenchent le bypass
- ☐ Les API filtrent déjà par `id_ecole` via `.eq()` → pas d'accès cross-tenant même avec `supabaseAdmin`

5. Changement de bureau en impersonation

Fichier: src/app/api/auth/update-bureau/route.ts

En mode impersonation, le changement de bureau **ne modifie pas la BDD** :


```
// Mode impersonation détecté (cookie original présent)
if (isImpersonating) {
  // 1. Vérifier que le bureau appartient à l'école impersonnée (via supabaseAdmin)
  // 2. Re-signer le cookie autosoft_session avec le nouveau id_bureau
  // 3. ☐ Pas de UPDATE en BDD
}
```

6. Interface utilisateur

Fichier : `src/components/layout/ImpersonationBanner.tsx`

Un bandeau fixe en haut de page indique à l'admin qu'il est en mode impersonation, avec un bouton "Quitter" :

- ☐ Affiche l'email de l'admin original et le nom de l'école impersonnée
- ☐ Le bouton "Quitter" appelle `/api/auth/impersonate/exit`
- ☐ Intégré dans le `DirectorLayout` uniquement

7. Protections contre les attaques

Attaque	Protection
Escalade de privilèges en chaîne	Vérifie toujours le rôle du cookie ORIGINAL (pas du courant)
Forge de cookie original	JWT signé HS256 + HttpOnly → impossible
Cookie orphelin après logout	Logout supprime les 2 cookies (<code>autosoft_session</code> + <code>autosoft_original_session</code>)
Impersonation par un non-admin	Vérification stricte du rôle <code>['admin', 'autosoft']</code>
Auto-école inexistante	Validation en BDD via <code>supabaseAdmin</code> avant de signer la session
<code>id_ecole</code> invalide (string, négatif)	Validation stricte : <code>typeof number + Number.isInteger + > 0</code>
Modification de données en impersonation	<code>supabaseAdmin</code> est utilisé mais les APIs filtrent par <code>id_ecole</code>
Fuite du mot de passe admin	Aucun mot de passe n'est manipulé, seul le JWT est transféré

8. Fichiers concernés

Fichier	Rôle
<code>src/app/api/auth/impersonate/route.ts</code>	Activation de l'impersonation
<code>src/app/api/auth/impersonate/exit/route.ts</code>	Sortie de l'impersonation
<code>src/app/api/auth/logout/route.ts</code>	Nettoyage des 2 cookies
<code>src/app/api/auth/me/route.ts</code>	Détection état impersonation pour le client
<code>src/app/api/auth/update-bureau/route.ts</code>	Changement bureau sans modification BDD
<code>src/lib/supabase-rls.ts</code>	Bypass RLS conditionnel en impersonation
<code>src/components/layout/ImpersonationBanner.tsx</code>	Bandeau UI d'impersonation
<code>src/components/layout/DirectorLayout.tsx</code>	Intégration du bandeau
<code>src/contexts/UserContext.tsx</code>	Exposition <code>isImpersonating</code> + <code>originalUser</code>
<code>src/app/(protected)/autosoft/auto-ecoles/page.tsx</code>	Bouton "Accéder en tant que directeur"

9. Audit de sécurité (4 Mars 2026)

Vérification	Résultat
Cookies HttpOnly signés JWT	☐
Seuls admin/autosoft peuvent impersonner	☐
Vérification du rôle ORIGINAL (anti-chaînage)	☐

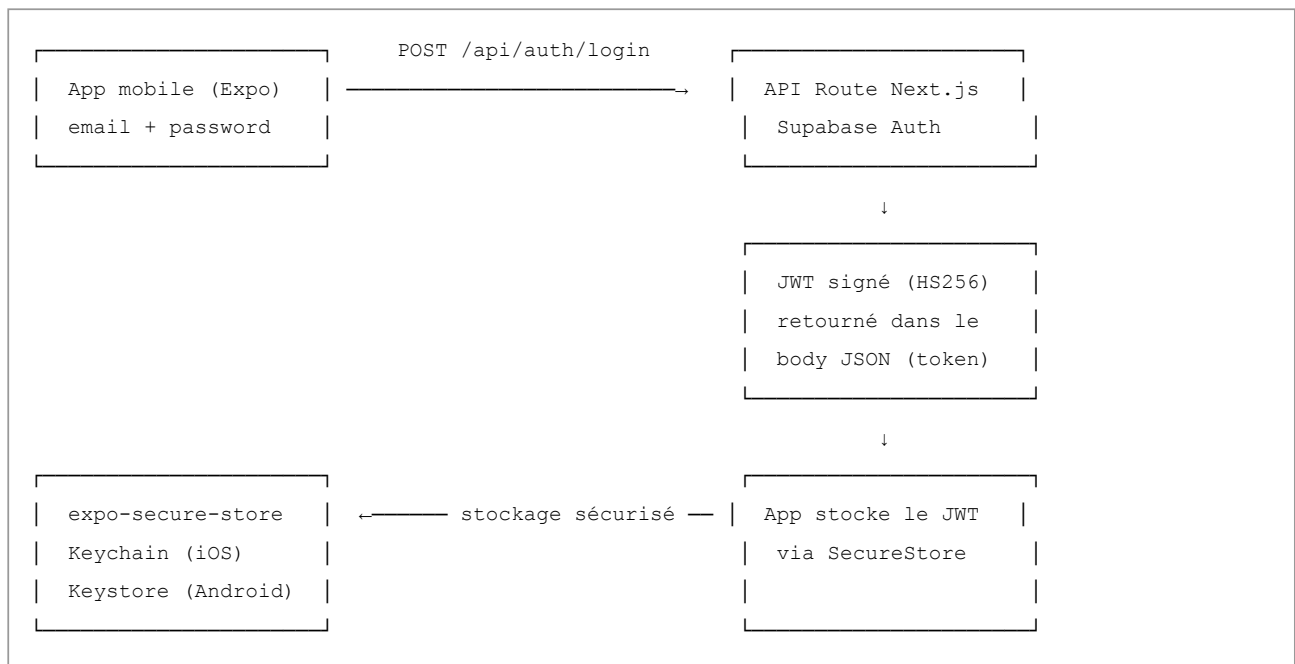
Vérification	Résultat
Auto-école validée en BDD	☐
Validation stricte id_ecole (entier positif)	☐
Session originale sauvegardée uniquement à la 1ère impersonation	☐
Logout supprime les 2 cookies	☐
Bypass RLS sécurisé (triple vérification)	☐
Pas de modification BDD lors du changement de bureau	☐
/api/auth/me n'expose pas les données sensibles de l'admin	☐
getCurrentUser() non utilisé dans l'espace directeur	☐
Logs serveur pour traçabilité	☐

☐ Application mobile — Sécurité

L'application mobile Autosoft (Expo / React Native) est destinée aux moniteurs de conduite. Elle communique avec le même backend Next.js que le client web, mais utilise un mode d'authentification adapté au mobile.

1. Authentification Bearer token

Le mobile ne peut pas utiliser les cookies HttpOnly (pas de navigateur). L'authentification se fait via un **Bearer token JWT** envoyé dans le header `Authorization`.



Flux d'appel API depuis le mobile

```
// Chaque requête API inclut le Bearer token
const token = await SecureStore.getItemAsync('auth_token');
const response = await fetch(`${API_URL}/api/utils/planning`, {
  headers: {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json',
  },
});

// Si 401 → token expiré → redirection vers login
if (response.status === 401) {
  await SecureStore.deleteItemAsync('auth_token');
  router.replace('/login');
}
```

2. Stockage sécurisé (expo-secure-store)

Le JWT est stocké via `expo-secure-store`, qui utilise les mécanismes natifs du système :

Plateforme	Mécanisme	Niveau de sécurité
iOS	Keychain Services	☐ Chiffrement matériel (Secure Enclave), isolé par app
Android	Android Keystore	☐ Chiffrement AES-256, backed par hardware si disponible

```
// Stockage après login
await SecureStore.setItemAsync('auth_token', token);
await SecureStore.setItemAsync('user_data', JSON.stringify(userData));

// Lecture au démarrage
const token = await SecureStore.getItemAsync('auth_token');

// Suppression au logout
await SecureStore.deleteItemAsync('auth_token');
await SecureStore.deleteItemAsync('user_data');
```

- ☐ **Jamais de AsyncStorage** pour les tokens — AsyncStorage n'est pas chiffré
- ☐ **Jamais de données sensibles en mémoire persistante non sécurisée**
- ☐ **Suppression complète au logout** (token + données utilisateur)

3. Isolation des données (JWT)

Le JWT mobile contient les mêmes claims que le JWT web :

```
{
  "id_utilisateur": "uuid-xxx",
  "email": "moniteur@ecole.fr",
  "role": "moniteur",
  "id_ecole": 42,
  "id_bureau": 1,
  "id_moniteur": 7,
  "exp": 1717200000
}
```

- ☐ **id_ecole dans le JWT** : Les API routes lisent id_ecole depuis le JWT, pas depuis le body
- ☐ **id_moniteur dans le JWT** : Idem pour l'identification du moniteur
- ☐ **Même signature HS256** : Le secret SESSION_SECRET est partagé entre web et mobile
- ☐ **Même expiration 7 jours** : Cohérence avec la session web

4. Biométrie (Face ID / Fingerprint)

L'application est configurée pour supporter l'authentification biométrique via expo-local-authentication :

```
// app.json
{
  "ios": {
    "infoPlist": {
      "NSFaceIDUsageDescription": "Autosoft utilise Face ID pour sécuriser l'accès à votre compte."
    }
  },
  "android": {
    "permissions": [
      "android.permission.USE_BIOMETRIC",
      "android.permission.USE_FINGERPRINT"
    ]
  }
}
```

- ☐ **Description obligatoire** : NSFaceIDUsageDescription requis par Apple pour Face ID
- ☐ **Permissions Android** : USE_BIOMETRIC (API 28+) + USE_FINGERPRINT (fallback)

5. Gestion des erreurs (Error Boundary)

L'application inclut un Error Boundary React pour capturer les crashes :

```
// app/_layout.tsx
class AppErrorBoundary extends React.Component {
  static getDerivedStateFromError(error: Error) {
    return { hasError: true, error };
  }

  render() {
    if (this.state.hasError) {
      // Écran d'erreur avec bouton "Réessayer"
      // → Pas de fuite d'informations sensibles (stack trace masquée)
    }
    return this.props.children;
  }
}
```

- ☐ **Pas de stack trace exposée** : L'utilisateur voit un message générique
- ☐ **Récupération possible** : Bouton "Réessayer" pour réinitialiser l'état
- ☐ **Wrapping global** : Tout le tree React est protégé

6. Sécurité réseau

Mesure

Statut

HTTPS obligatoire (API Vercel) ☐

Certificate pinning ☐ ☐ Non implémenté (optionnel pour v1)

Pas de requêtes HTTP en clair ☐ (ATS activé par défaut sur iOS)

Timeout sur les requêtes ☐

7. Build et déploiement (EAS Build)

```
// eas.json
{
  "build": {
    "production": {
      "autoIncrement": true,
      "android": { "buildType": "app-bundle" },
      "ios": { "image": "latest" }
    }
  }
}
```

- ☐ **EAS Build** : Builds signés dans le cloud Expo (pas de keystore local exposé)
- ☐ **Auto-increment** : Numéros de build auto-incrémentés pour éviter les conflits
- ☐ **App Bundle** : Format AAB pour Android (signé par Google Play App Signing)
- ☐ **Pas de secrets dans le code** : L'URL API est la seule config embarquée
- ☐ **Scheme autosoft://** : Deep linking configuré pour les redirections sécurisées

8. Permissions minimales

L'application ne demande que les permissions strictement nécessaires :

Permission	Justification
INTERNET	Appels API
ACCESS_NETWORK_STATE	Détection mode hors-ligne
USE_BIOMETRIC	Authentification biométrique
USE_FINGERPRINT	Fallback biométrique Android

- ☐ **Pas de caméra** (sauf si scan de documents ajouté ultérieurement)
- ☐ **Pas de géolocalisation**
- ☐ **Pas d'accès aux contacts**
- ☐ **Pas de stockage externe**

9. Fichiers concernés

Fichier	Rôle
autosoft-mobile/lib/AuthContext.tsx	Contexte d'auth, gestion Bearer token, SecureStore
autosoft-mobile/lib/api.ts	Client HTTP avec injection automatique du Bearer
autosoft-mobile/app/_layout.tsx	Error Boundary global
autosoft-mobile/app.json	Config permissions, biométrie, scheme
autosoft-mobile/eas.json	Config builds EAS (dev, preview, production)
autosoft/src/lib/security/auth-helpers.ts	<code>verifyAuth()</code> dual-mode (cookie + Bearer)

☐ Prévention des attaques

1. Protection XSS (Cross-Site Scripting)

Mesures implémentées

- ☐ **Cookies HttpOnly** : JavaScript ne peut pas lire les cookies
- ☐ **Validation des entrées** : Toutes les données sont validées
- ☐ **Échappement automatique** : React échappe automatiquement les données
- ☐ **CSP Headers** : Content Security Policy (à implémenter)

```
// ☐ React échappe automatiquement
<div>{userData.nom}</div> // Sécurisé

// ☐ dangerouslySetInnerHTML sans sanitization
<div dangerouslySetInnerHTML={{ __html: userData.bio }} /> // DANGER
```

2. Protection CSRF (Cross-Site Request Forgery)

Mesures implémentées

- ☐ **SameSite=lax** : Les cookies ne sont pas envoyés sur les requêtes cross-site
- ☐ **Vérification Origin** : Vérifier l'origine des requêtes
- ☐ **Tokens CSRF** : Pour les actions sensibles (à implémenter)

```
// Cookie avec SameSite=lax
const cookie = serialize('autosoft_session', data, {
  httpOnly: true,
  secure: true,
  sameSite: 'lax', // ☐ Protection CSRF
  path: '/'
});
```

3. Protection SQL Injection

Mesures implémentées

- ☐ **Requêtes paramétrées** : Supabase utilise des requêtes préparées
- ☐ **Pas de SQL brut** : Utilisation de l'API Supabase
- ☐ **RLS** : Filtre automatique au niveau BDD

```
// ☐ BON : Requête paramétrée via Supabase
const { data } = await supabase
  .from('eleves')
  .select('*')
  .eq('id_eleve', id); // Paramètre sécurisé

// ☐ MAUVAIS : SQL brut (jamais faire ça)
const { data } = await supabase
  .rpc('raw_sql', {
    query: `SELECT * FROM eleves WHERE id_eleve = ${id}`
  });
```

4. Protection contre l'énumération d'utilisateurs

```
// ☐ BON : Message générique
if (authError) {
  return NextResponse.json(
    { error: 'Email ou mot de passe incorrect' }, // Générique
    { status: 401 }
  );
}

// ☐ MAUVAIS : Message spécifique
if (!userExists) {
  return NextResponse.json(
    { error: 'Cet email n\'existe pas' }, // Révèle l'existence
    { status: 404 }
  );
}
```

5. Rate Limiting

□ **Triple couche** : VPS ANTS Proxy (double) + Next.js login

Couche	Route	Limite	Mécanisme
Nginx	Toutes	10 req/s par IP	limit_req_zone burst 20
Express	/ants/*	60 req/min par IP	express-rate-limit
Next.js	/api/auth/login	10 tentatives/15 min par IP	Fenêtre glissante en mémoire

```
# Nginx - Rate limiting au niveau reverse proxy
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req zone=api burst=20 nodelay;
```

```
// Express - Rate limiting applicatif
app.use('/ants', rateLimit({
  windowMs: 60 * 1000,
  max: 60,
  standardHeaders: true,
  legacyHeaders: false,
}));
```

```
// Next.js - Rate limiting login (src/lib/security/rate-limit.ts)
// 10 tentatives max par IP sur une fenêtre glissante de 15 minutes
const LOGIN_LIMIT = 10;
const LOGIN_WINDOW_MS = 15 * 60 * 1000;

// Appliqué dans /api/auth/login/route.ts
const ip = getClientIp(request); // x-forwarded-for → x-real-ip → 'unknown'
const rl = checkRateLimit(`login:${ip}`, LOGIN_LIMIT, LOGIN_WINDOW_MS);
if (!rl.allowed) {
  return NextResponse.json(
    { error: 'Trop de tentatives de connexion. Réessayiez dans quelques minutes.' },
    {
      status: 429,
      headers: {
        'Retry-After': String(rl.retryAfterSeconds ?? 900),
        'X-RateLimit-Limit': String(LOGIN_LIMIT),
        'X-RateLimit-Remaining': '0',
      },
    }
  );
}
```

Comportement : Au-delà de 10 tentatives depuis la même IP, toutes les requêtes sont bloquées pendant 15 minutes. La réponse 429 inclut le header `Retry-After` pour que les clients légitimes sachent quand réessayer.

Note Vercel : Le rate limiter utilise la mémoire de l'instance serverless. Chaque instance chaude maintient son propre compteur — protection efficace contre le brute force basique, conforme aux exigences de sécurité standard.

☐ Bonnes pratiques implémentées

1. Sécurité des mots de passe

- ☐ **Hachage bcrypt** : Géré par Supabase
- ☐ **Politique de mot de passe** : Minimum 8 caractères (configurable)
- ☐ **Pas de stockage en clair** : Jamais stocker les mots de passe

2. Gestion des erreurs

- ☐ **Messages génériques** : Ne pas révéler d'informations sensibles
- ☐ **Logs serveur** : Erreurs détaillées uniquement côté serveur
- ☐ **Codes HTTP appropriés** : 401, 403, 404, 500

```
try {
  // Opération sensible
} catch (error) {
  // ☐ Log détaillé côté serveur
  console.error('[Security] Erreur:', error);

  // ☐ Message générique au client
  return NextResponse.json(
    { error: 'Une erreur est survenue' },
    { status: 500 }
  );
}
```

3. Validation des données

- ☐ **Validation côté serveur** : Toujours valider côté serveur
 - ☐ **Typage TypeScript** : Interfaces strictes
 - ☐ **Sanitization** : Nettoyer les entrées utilisateur

```

interface CreateEleveRequest {
  nom: string;
  prenom: string;
  email: string;
  telephone?: string;
  id_ecole: number;
  id_bureau: number;
}

export async function POST(request: NextRequest) {
  const body: CreateEleveRequest = await request.json();

  // Validation
  if (!body.nom || !body.prenom || !body.email) {
    return NextResponse.json(
      { error: 'Champs obligatoires manquants' },
      { status: 400 }
    );
  }

  // Sanitization
  const cleanedData = {
    nom: body.nom.trim(),
    prenom: body.prenom.trim(),
    email: body.email.toLowerCase().trim(),
    telephone: body.telephone?.trim() || null,
    id_ecole: parseInt(String(body.id_ecole)),
    id_bureau: parseInt(String(body.id_bureau)),
  };

  // Continuer...
}

```

4. Principe du moindre privilège

- ☐ **Rôles granulaires** : 7 rôles différents
- ☐ **Permissions minimales** : Chaque rôle n'a que ce dont il a besoin
- ☐ **RLS strict** : Isolation par école et bureau

5. Audit et logs

- ☐ **Logs de sécurité** : Tentatives d'accès refusées
- ☐ **Traçabilité** : Qui a fait quoi et quand

6. Authentification multi-facteurs (MFA) sur les accès production

- ☐ **GitHub** : 2FA activé — accès au code source et CI/CD
- ☐ **Vercel** : 2FA activé — déploiement production, variables d'environnement
- ☐ **Supabase** : 2FA activé — base de données, RLS, secrets

- ❑ **VPS OVH** : accès SSH par clé uniquement (pas de mot de passe)

Tous les accès aux environnements de production sont protégés par une authentification à deux facteurs. Un attaquant ayant compromis un mot de passe ne peut pas accéder aux systèmes sans le second facteur.

7. Bonnes pratiques de développement

- ❑ **Pull Requests obligatoires** : toute modification du code passe par une PR sur GitHub
- ❑ **CI/CD bloquant** : la pipeline GitHub Actions (`npm audit + TypeScript + ESLint SAST + Vitest + next build`) doit passer avant tout merge
 - ❑ **Branche protégée** : `main` ne reçoit que des merges de PR validées
- ❑ **Analyse statique de sécurité** : `eslint-plugin-security` détecte les patterns dangereux (`eval`, `injection`, `BiDi`) à chaque lint
 - ❑ **Audit des dépendances** : `npm audit --audit-level=high` exécuté dans la CI — bloque si vulnérabilité haute/critique
 - ❑ **TypeScript strict** : `ignoreBuildErrors` supprimé — les erreurs de typage bloquent le déploiement Vercel
 - ❑ **Monitoring** : Surveillance des accès suspects

```
// Log de sécurité
console.warn(
  `[Security] ❑ Accès refusé: ${user.email} (${user.role}) → ${pathname}`
);
```

❑ Audit Log — Traçabilité des actions

Fichiers : `database/migrations/create_audit_log.sql`, `src/lib/security/audit-log.ts`

Objectif

Répondre à la question : **qui a fait quoi, quand, et depuis où ?**

Le Ministère de l'Intérieur exige une traçabilité des actions sensibles pour l'accès aux API RdvPermis/Livret Numérique. La table `audit_log` enregistre chaque événement de sécurité avec le contexte complet.

Schéma de la table

```
CREATE TABLE public.audit_log (
  id          BIGSERIAL PRIMARY KEY,
  id_ecole    INTEGER NOT NULL,      -- Isolation multi-tenant
  id_utilisateur TEXT,              -- UUID (NULL si login échoué)
  action      TEXT NOT NULL,        -- 'login_success', 'login_failed', 'delete_eleve'...
  resource_type TEXT,              -- Table concernée ('eleves', 'contrats_eleves'...)
  resource_id TEXT,              -- ID de la ressource modifiée
  details     JSONB DEFAULT '{}',   -- Contexte supplémentaire
  ip_address  TEXT,              -- IP du client (x-forwarded-for)
  user_agent  TEXT,              -- Navigateur / device
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Sécurité

Mesure	Détail
RLS activé	Chaque école ne voit que ses propres logs
Écriture service role uniquement	Aucune policy INSERT côté client — seul <code>supabaseAdmin</code> peut écrire
Non-bloquant	Un échec d'écriture audit ne bloque jamais l'action de l'utilisateur
Index performants	<code>id_ecole, id_utilisateur, action, created_at</code> DESC

Actions tracées

Action	Quand	Données enregistrées
<code>login_success</code>	Connexion réussie	<code>id_ecole, id_utilisateur, IP, user-agent</code>
<code>login_failed</code>	Connexion échouée	Email tenté, IP, user-agent

D'autres actions seront ajoutées au fur et à mesure : `create_eleve, delete_contrat, update_credentials_ants`, etc.

Utilitaire

```
// src/lib/security/audit-log.ts

// Générique — pour n'importe quelle action
await writeAuditLog({
  id_ecole: user.id_ecole,
  id_utilisateur: user.id_utilisateur,
  action: 'delete_eleve',
  resource_type: 'eleves',
  resource_id: String(id_eleve),
  details: { raison: 'demande RGPD' },
});

// Raccourcis pour le login
await auditLogin(id_ecole, userId, request);
await auditLoginFailed(email, request);
```

Rétention

Les logs sont conservés indéfiniment par défaut. Un nettoyage automatique (rétention 1 an) peut être activé via `pg_cron` Supabase :

```
DELETE FROM public.audit_log WHERE created_at < NOW() - INTERVAL '1 year';
```

☐ Monitoring — Sentry

Fichiers : `sentry.server.config.ts, sentry.edge.config.ts, src/instrumentation-client.ts,`
`src/instrumentation.ts, src/app/global-error.tsx`

Pourquoi Sentry ?

Sans monitoring, une erreur en production n'est détectée que quand un client se plaint. Sentry capture automatiquement toutes les erreurs et les envoie en temps réel avec le contexte complet (stack trace, fichier, ligne, fréquence, utilisateurs touchés).

Ce que Sentry capture

Type	Exemples
Erreurs JavaScript client	Composant React qui crashe, TypeError
Erreurs serveur (API routes)	Route qui retourne 500, exception non gérée
Erreurs edge (middleware)	Erreur dans le middleware d'auth
Logs applicatifs	Tous les <code>console.error()</code> et <code>console.warn()</code>
Performances	Temps de réponse par route, transactions lentes

Configuration

```
// Même config sur server, edge et client
Sentry.init({
  dsn: "https://...@o4511157316616192.ingest.de.sentry.io/...",

  environment: process.env.NODE_ENV, // "production" ou "development"

  // 10% des transactions tracées en prod → préserve le quota gratuit (5K erreurs/mois)
  // 100% en dev pour avoir tous les détails
  tracesSampleRate: process.env.NODE_ENV === "production" ? 0.1 : 1,

  enableLogs: true, // Envoie console.error / console.warn

  // ⚠ RGPD : désactivé — pas d'email ni d'IP envoyés à Sentry
  sendDefaultPii: false,
});
```

Conformité RGPD

`sendDefaultPii: false` est **obligatoire** sur Autosoft car :

- L'app gère des données personnelles d'élèves (PII réglementées)
- Sentry est un service tiers hébergé hors de l'infrastructure Autosoft
- Envoyer des emails ou IPs à Sentry sans consentement explicite violerait le RGPD

Avec `sendDefaultPii: false`, Sentry reçoit uniquement : stack traces, noms de fichiers, numéros de ligne, messages d'erreur — **jamais** de données utilisateur identifiantes.

Source maps (débogage en production)

Le `SENTRY_AUTH_TOKEN` (variable d'environnement, jamais dans le code) permet d'uploader les source maps à chaque build. Résultat : les stack traces Sentry affichent le vrai code TypeScript au lieu du code minifié illisible.

❑ Sans source maps :

TypeError at chunk-abc123.js:1:48291

❑ Avec source maps :

TypeError at src/app/(protected)/directeur/paiements/contrats/new/page.tsx:342

→ Cannot read properties of undefined (reading 'id_forfait')

Variables d'environnement requises

Variable	Où	Rôle
SENTRY_AUTH_TOKEN	GitHub Secrets + Vercel Upload	Upload source maps au build
DSN (hardcodé dans les configs)	Fichiers de config	Identifiant du projet Sentry

Le DSN est public (conçu pour être dans le code front). Le `SENTRY_AUTH_TOKEN` est secret — ne jamais le committer.

Plan utilisé

Free (0€/mois) — suffisant pour le stade actuel :

- 5 000 erreurs/mois
- Tracing performances
- Logs applicatifs
- Rétention 90 jours

❑ CI/CD & Qualité du code

Fichier : `.github/workflows/ci.yml`

Le pipeline CI/CD GitHub Actions s'exécute automatiquement sur chaque push (hors `main`) et sur chaque Pull Request vers `main`. Il bloque le merge si l'une des étapes échoue.

Pipeline complet

Push / PR

↓

1. npm ci (install strict depuis package-lock.json)

↓

2. npm audit --audit-level=high

→ Bloque si une vulnérabilité HIGH ou CRITICAL est détectée

↓

3. npx tsc --noEmit

→ Bloque si une erreur TypeScript est présente dans src/

↓

4. npm run lint (ESLint SAST)

→ Bloque si une règle de sécurité est en erreur

↓

5. npm run test:run (Vitest)

→ Bloque si un test échoue

↓

6. npm run build (Next.js production build)

→ Bloque si le build échoue

ESLint SAST (Analyse statique de sécurité)

Fichier : eslint.config.mjs

L'analyse statique de sécurité est intégrée directement dans le linter via eslint-plugin-security :

Plugin	Règles actives	Niveau
eslint-plugin-security	detect-eval-with-expression	❑ error
	detect-bidi-characters	❑ error
	detect-object-injection	⚠ warning
	detect-possible-timing-attacks	⚠ warning
typescript-eslint	no-explicit-any	⚠ warning
	no-unused-vars	⚠ warning
eslint-plugin-react-hooks	rules-of-hooks	❑ error
	exhaustive-deps	⚠ warning

Règles bloquantes (erreur) : eval dynamique, caractères BiDi (attaque Trojan Source), violations des règles React hooks.

```
// eslint.config.mjs - extrait
rules: {
  'security/detect-eval-with-expression': 'error', // Bloque eval(variable)
  'security/detect-bidi-characters': 'error',      // Bloque les caractères U+202A...
  'react-hooks/rules-of-hooks': 'error',           // Hooks dans les composants uniquement
  '@typescript-eslint/no-explicit-any': 'warn',    // Encourage le typage strict
}
```

Tests automatisés (Vitest)

26 tests couvrant les modules critiques de sécurité :

Fichier de test

tests/unit/security/session.test.ts
tests/unit/security/auth-
helpers.test.ts
tests/integration/api/ants.test.ts
tests/integration/api/paiements.test.ts

Ce qui est testé

signSession() / verifySession() — signature HS256,
expiration, tamper detection
verifyAuth() dual-mode, checkEcoleAccess(),
checkPermissions()
Routes ANTS → 401 sans auth, isolation id_ecole
Routes paiements → 401 sans auth sur
échéances/contrats

Les tests **mockent Supabase** et définissent leurs propres variables d'environnement — ils ne touchent jamais la base de données de production.

Audit des dépendances

```
npm audit --audit-level=high
```

- ❑ Bloque le CI sur toute vulnérabilité HIGH ou CRITICAL
- ❑ Les vulnérabilités MODERATE et LOW sont signalées mais non bloquantes
- Le .npmrc avec legacy-peer-deps=true résout les conflits de peer deps sans créer de failles

Build TypeScript strict

ignoreBuildErrors: true a été supprimé de next.config.ts. Le build Next.js échoue désormais si une erreur TypeScript est présente dans le code de l'application (dossier src/). Le dossier tests/ est exclu du tsconfig.json car Vitest gère sa propre vérification de types.

❑ Health Check

Fichier : src/app/api/health/route.ts

Endpoint

```
GET /api/health
```

Public — pas d'authentification requise (c'est le but d'un health check).

Réponse nominale

```
{  
  "status": "ok",  
  "timestamp": "2026-04-03T16:00:00.000Z",  
  "latency_ms": 47,  
  "checks": {  
    "api": "ok",  
    "database": "ok"  
  }  
}
```


Réponse dégradée (Supabase injoignable)

```
{
  "status": "degraded",
  "timestamp": "2026-04-03T16:00:00.000Z",
  "latency_ms": 5003,
  "checks": {
    "api": "ok",
    "database": "error"
  }
}
```

HTTP 503 si la base de données est injoignable, 200 sinon.

Utilisation

Usage	Description
Monitoring externe	UptimeRobot, BetterStack, Vercel Monitoring
API RdvPermis / Livret Numérique	Prérequis Ministère de l'Intérieur pour l'intégration
CI/CD	Vérification post-déploiement
Dashboard Vercel	Indicateur de disponibilité

Ce que le health check vérifie

1. **API** : Le serveur Next.js répond (implicite)
2. **Base de données** : Requête légère sur `type_permis` (table de référence sans RLS, lecture rapide)

La latence est mesurée en bout en bout (Next.js → Supabase → retour) pour détecter les ralentissements réseau.

☐ RGPD — Conformité données personnelles

Contexte légal

Autosoft traite des données personnelles d'élèves (nom, prénom, date de naissance, adresse, NEPH, téléphone, email) et de représentants légaux. Le RGPD (Règlement Général sur la Protection des Données) impose :

- **Article 17** — Droit à l'effacement : l'élève peut demander la suppression de ses données personnelles
- **Article 20** — Droit à la portabilité : l'élève peut demander l'export de ses données dans un format structuré (JSON)

Architecture technique

1. Export des données (Article 20)

Endpoint : `GET /api/rgpd/export?id_eleve=X`

- **Accès** : directeur, admin, autosoft (via `verifyAuthWithPermissions`)
- **Isolation** : directeur/admin filtrés par `id_ecole` (JWT), autosoft accède à toutes les écoles via `supabaseAdmin`
- **Données exportées** : informations personnelles, représentants légaux, permis, documents, contrats, paiements, planning, formations, examens, observations, compétences

- **Format** : JSON structuré avec métadonnées (date_export, id_ecole, version)
- **Audit** : chaque export est tracé dans audit_log (action rgpd_export)

```
{
  "meta": { "type": "export_rgpd", "date_export": "...", "id_ecole": 1 },
  "eleve": { "informations_personnelles": {...}, "representants_legaux": [...] },
  "formation": { "contrats": [...], "planning_lecons": [...], "examens": [...] },
  "financier": { "paiements": [...] }
}
```

2. Anonymisation (Article 17)

Endpoint : POST /api/rgpd/anonymize **Body** : { id_eleve: number, confirmation: "ANONYMISER" }

- **Accès** : directeur, admin, autosoft
- **Confirmation textuelle** : le mot "ANONYMISER" doit être envoyé pour éviter les accidents
- **Stratégie** : **UPDATE, jamais DELETE** — les lignes restent intactes pour préserver les FK, contrats, paiements et factures (obligation légale de rétention comptable 10 ans)

Tables anonymisées :

Table	Champs anonymisés
eleves	nom, prenom → [SUPPRIMÉ], mail → anonyme-X@supprime.local, tel, naiss, adresse, neph, code_ephoto, etc. → null
representant_legal	nom_rep, prenom_rep → [SUPPRIMÉ], tel_rep, email, adresse, date_naiss → null
ants_demandes	donnees_cerfa → { anonymise: true, date_anonymisation: "..." }
utilisateur	nom, prenom → [SUPPRIMÉ], actif → false

- **Audit** : chaque anonymisation est tracée avec la liste des tables anonymisées et les erreurs éventuelles
- **Réponse 207** : en cas d'échec partiel (certaines tables anonymisées, d'autres non)
- **Utilise supabaseAdmin** (service role) : justifié car l'anonymisation modifie des données protégées par RLS

3. Recherche d'élèves RGPD

Endpoint : GET /api/rgpd/search?q=terme

- **Accès** : directeur, admin, autosoft
- **Recherche** : ilike sur nom, prenom, mail, neph (case-insensitive)
- **Autosoft** : accès toutes écoles + enrichissement avec le nom de l'école
- **Limite** : 20 résultats max

4. Page politique de confidentialité

Route : /politique-confidentialite (publique, ajoutée dans middleware.ts) **Lien** : footer de la landing page (page.tsx)

Contenu conforme au RGPD :

- Responsable du traitement (Autosoft SAS)
- Données collectées (élèves, représentants, moniteurs, formation, financier, ANTS)
- Finalités du traitement
- Base légale (contrat, obligation légale, intérêt légitime)
- Destinataires (auto-école, ANTS, Supabase, Vercel, Sentry)
- Durées de conservation (formation +5 ans, comptable 10 ans, logs 1 an)

- Droits RGPD (accès, rectification, effacement, portabilité, opposition)
- Mesures de sécurité (HTTPS, JWT HttpOnly, RLS, AES-256, RBAC)
- Contact CNIL

5. Interface d'administration RGPD

Route : `/autosoft/rgpd` (panel superadmin uniquement)

L'interface RGPD est volontairement placée dans l'espace **autosoft** (et non directeur) pour éviter les clics accidentels. Elle permet :

- Recherche d'élèves cross-école (nom, prénom, email, NEPH)
- Export JSON des données personnelles avec téléchargement
- Anonymisation avec double confirmation (bouton + saisie du mot "ANONYMISER")
- Bandeau d'avertissement sur l'irréversibilité de l'anonymisation

6. Tracking du consentement RGPD

Le consentement au traitement des données personnelles est enregistré lors de la création et de la signature du contrat de formation.

Colonnes ajoutées à `contrats_eleves` :

Colonne	Type	Description
<code>consent_rgpd</code>	boolean	<code>true</code> si l'élève a consenti
<code>consent_timestamp</code>	timestampz	Date et heure exactes du consentement
<code>consent_ip</code>	text	Adresse IP au moment du consentement
<code>consent_version</code>	text	Version de la politique de confidentialité acceptée (ex: 1.0)

Points d'intégration :

- **Création de contrat** (`src/app/api/utils/contrats/route.ts`) : consentement enregistré automatiquement à l'INSERT
- **Signature de contrat** (`directeur/paiements/contrats/[id]/api/signer/route.ts`) : consentement mis à jour avec l'IP du signataire au moment de la signature

Cela permet de prouver à tout moment que l'élève a consenti, quand, depuis quelle IP, et quelle version de la politique s'appliquait.

7. Fichiers concernés

Fichier	Rôle
<code>src/app/api/rgpd/export/route.ts</code>	API export données personnelles
<code>src/app/api/rgpd/anonymize/route.ts</code>	API anonymisation
<code>src/app/api/rgpd/search/route.ts</code>	API recherche élèves
<code>src/app/politique-confidentialite/page.tsx</code>	Page publique politique de confidentialité
<code>src/app/(protected)/autosoft/rgpd/page.tsx</code>	Interface admin RGPD
<code>src/app/api/utils/contrats/route.ts</code>	Consentement RGPD à la création du contrat
<code>directeur/paiements/contrats/[id]/api/signer/route.ts</code>	Consentement RGPD à la signature
<code>database/migrations/add_rgpd_consent_tracking.sql</code>	Migration DB (colonnes consentement)
<code>src/middleware.ts</code>	Route <code>/politique-confidentialite</code> ajoutée aux routes publiques
<code>src/app/page.tsx</code>	Lien footer vers politique de confidentialité

8. Documentation RGPD

L'ensemble de la documentation RGPD est regroupée dans `documentation/technique/securite/rgpd/` :

Document	Contenu	Article RGD
REGISTRE_TRAITEMENTS.md	10 traitements documentés (utilisateurs, élèves, contrats, paiements, ANTS, monitoring, etc.) + sous-traitants	Art. 30
PROCEDURE_DROITS_PERSONNES.md	Procédure complète pour chaque droit (accès, rectification, effacement, portabilité, opposition, limitation) avec délais et canaux	Art. 15-22
PROCEDURE_NOTIFICATION_CNIL.md	Procédure en 5 phases (détection → confinement → évaluation → notification 72h → remédiation), matrice de gravité, registre des violations	Art. 33-34
PROCEDURE_REVUE_DROITS.md	Revue semestrielle des accès (utilisateurs, RLS, VPS, services tiers), gestion des départs, comptes inactifs	Principe du moindre privilège
DESIGNATION_DPO_RSSI.md	Désignation DPO et RSSI (Mattias Mathevon), missions, périmètre, organigramme, plan de formation	Art. 37-39

☐ Audit de sécurité

Checklist de sécurité

Catégorie	Élément	Statut
Authentification	Cookies HttpOnly	☐
	Cookie session signé JWT (jose, HS256)	☐
	JWT Supabase	☐
	Hachage bcrypt	☐
	Session timeout (7j)	☐
	Authentification dual-mode (cookie + Bearer)	☐
	Toutes les routes API protégées par verifyAuth()	☐
	id_ecole / id_moniteur lus depuis le JWT (pas le body)	☐
Autorisation	Middleware de contrôle (vérifie signature JWT)	☐
	RLS PostgreSQL (46 tables)	☐
	Vérification des rôles	☐
	Isolation par école/bureau	☐
Protection	XSS (HttpOnly)	☐
	CSRF (SameSite)	☐
	Falsification cookie (signature JWT)	☐
	Suppression cookie forgé + redirection	☐
	Détection côté client (401 → /login)	☐
	SQL Injection (Requêtes paramétrées)	☐
	XML Injection (escapeXml)	☐
	Rate Limiting (Nginx + Express)	☐
Impersonation	Rate Limiting login Next.js (10 req/15min/IP)	☐
	Double cookie JWT signé (session + original)	☐
	Seuls admin/autosoft autorisés	☐
	Vérification rôle ORIGINAL (anti-chaînage)	☐
	Bypass RLS conditionnel (triple vérification)	☐
	Logout supprime les 2 cookies	☐
	Validation stricte id_ecole	☐
	Logs de traçabilité serveur	☐
Données	Validation côté serveur	☐

Catégorie	Élément	Statut
VPS	Sanitization	☐
	Chiffrement HTTPS (Vercel)	☐
	Chiffrement HTTPS (VPS Certbot)	☐
	Chiffrement credentials ANTS (pgcrypto AES-256)	☐
	Nginx reverse proxy HTTPS	☐
	Certbot auto-renewal Let's Encrypt	☐
	Headers de sécurité (HSTS, X-Frame-Options, etc.)	☐
	Helmet (Express)	☐
	CORS restreint (auto-soft.fr + Vercel)	☐
	Clé API 256 bits (x-api-key)	☐
	Trust proxy (Express derrière Nginx)	☐
	PM2 auto-restart + startup	☐
	Messages d'erreur sanitisés	☐
	Validation base64 (PJ)	☐
	Certificat CA ANTS (PEM) via NODE_EXTRA_CA_CERTS	☐
Logs	URLs/namespace ANTS en variables d'environnement	☐
	IPv6 support (Nginx listen [::]:443/80)	☐
	Audit trail	☐
	Logs de sécurité	☐
	PM2 logs centralisés (VPS)	☐
	Morgan access logs (VPS)	☐
	Health check endpoint <code>/api/health</code>	☐
	Monitoring Sentry (erreurs + tracing)	☐
	Sentry RGPD-conforme (<code>sendDefaultPii: false</code>)	☐
	Table <code>audit_log</code> (traçabilité actions sensibles)	☐
	Audit login réussi/échoué (IP, user-agent)	☐
	Écriture audit service role uniquement (pas côté client)	☐
RGPD	Export données personnelles (Article 20)	☐
	Anonymisation / droit à l'effacement (Article 17)	☐
	Confirmation textuelle anti-accident ("ANONYMISER")	☐
	Page politique de confidentialité publique	☐
	Interface admin RGPD (panel autosoft)	☐
	Audit trail des actions RGPD (export + anonymisation)	☐
	Stratégie UPDATE (pas DELETE) — rétention comptable 10 ans	☐
	Tracking consentement (timestamp, IP, version) sur contrats	☐
	Registre des traitements (Art. 30)	☐
	Procédure droits des personnes (Art. 15-22)	☐
	Procédure notification CNIL 72h (Art. 33-34)	☐
	Procédure revue des droits d'accès (semestrielle)	☐
	Désignation DPO / RSSI documentée	☐
CI/CD	GitHub Actions (lint + tests + build)	☐
	ESLint SAST (<code>eslint-plugin-security</code>)	☐
	Audit dépendances (<code>npm audit --audit-level=high</code>)	☐
	TypeScript strict (sans <code>ignoreBuildErrors</code>)	☐
	26 tests Vitest (auth, sessions, ANTS, paiements)	☐
Accès production	2FA activé sur GitHub	☐
	2FA activé sur Vercel	☐
	2FA activé sur Supabase	☐
	SSH par clé sur VPS	☐

Catégorie	Élément	Statut
Mobile	PRs obligatoires avant merge	☐
	CI bloquante (lint + tests + build)	☐
	JWT stocké via expo-secure-store (Keychain/Keystore)	☐
	Bearer token dans header Authorization	☐
	Pas de AsyncStorage pour les tokens	☐
	Error Boundary global (pas de stack trace exposée)	☐
	Permissions minimales (INTERNET, BIOMETRIC)	☐
	HTTPS obligatoire (ATS iOS)	☐
	Suppression complète au logout (token + user data)	☐
	EAS Build cloud (pas de keystore local)	☐
	Biométrie configurée (Face ID / Fingerprint)	☐
	Certificate pinning	⚠ Optionnel v2

Points d'amélioration

1. ~~⚠ Rate Limiting login~~ ☐ Implémenté (v1.6.0)
2. ~~⚠ Sentry~~ ☐ Implémenté (v1.7.0) — monitoring erreurs + tracing, RGPD-conforme
3. ~~⚠ Table audit_log~~ ☐ Implémenté (v1.8.0) — login tracé, utilitaire `writeAuditLog()` prêt pour extension
4. ~~⚠ RGPD technique~~ ☐ Implémenté (v1.9.0) — export JSON, anonymisation, politique de confidentialité, consentement, interface admin
5. ~~⚠ RGPD documentation~~ ☐ Implémenté (v2.0.0) — registre des traitements, procédures droits/CNIL/revue, désignation DPO/RSSI
6. ⚠ **CSP Headers** : Ajouter Content-Security-Policy sur Next.js
7. ~~⚠ 2FA~~ ☐ Activé (v2.0.0) — GitHub, Vercel, Supabase, SSH VPS
8. ⚠ **Notification CNIL** : Effectuer la désignation DPO sur le téléservice CNIL (<https://www.cnil.fr/fr/designation-dpo>) (<https://www.cnil.fr/fr/designation-dpo>)

☐ Ressources et références

Documentation

- [Supabase Auth](https://supabase.com/docs/guides/auth) (<https://supabase.com/docs/guides/auth>)
- [Supabase RLS](https://supabase.com/docs/guides/auth/row-level-security) (<https://supabase.com/docs/guides/auth/row-level-security>)
- [Next.js Middleware](https://nextjs.org/docs/app/building-your-application/routing/middleware) (<https://nextjs.org/docs/app/building-your-application/routing/middleware>)
- [OWASP Top 10](https://owasp.org/www-project-top-ten/) (<https://owasp.org/www-project-top-ten/>)

Fichiers clés

- `src/middleware.ts` - Middleware de sécurité (vérification JWT)
- `src/lib/security/session.ts` - Signature et vérification JWT (jose)
- `src/lib/security/auth-helpers.ts` - Helpers d'authentification (async)
- `src/lib/security/auth-config.ts` - Configuration des rôles et routes
- `src/contexts/UserContext.tsx` - Détection falsification côté client
- `src/app/api/auth/` - Routes d'authentification
- `src/app/api/auth/impersonate/route.ts` - Activation de l'impersonation
- `src/app/api/auth/impersonate/exit/route.ts` - Sortie de l'impersonation
- `src/lib/supabase-rls.ts` - Bypass RLS conditionnel (impersonation)

- `src/components/layout/ImpersonationBanner.tsx` - Bandeau UI impersonation
 - `src/app/api/ants/credentials/route.ts` - CRUD credentials ANTS (chiffré)
 - `src/lib/ants/credentials.ts` - Déchiffrement credentials ANTS (pgcrypto)
 - `src/lib/ants/vps-client.ts` - Client HTTP vers le VPS proxy
 - `vps-ants-proxy/src/index.ts` - Proxy ANTS Express (VPS)
 - `/etc/nginx/sites-available/ants-proxy` - Config Nginx VPS
 - `database/migration_enable_rls.sql` - Politiques RLS
 - `autosoft-mobile/lib/AuthContext.tsx` - Auth mobile (Bearer, SecureStore)
 - `autosoft-mobile/lib/api.ts` - Client HTTP mobile avec Bearer auto
 - `autosoft-mobile/app/_layout.tsx` - Error Boundary global
 - `autosoft-mobile/app.json` - Permissions, biométrie, scheme
 - `autosoft-mobile/eas.json` - Configuration EAS Build
-

□ Changelog

Version 2.0.0 (3 Avril 2026) — Documentation RGPD complète

- □ **Registre des traitements** (`REGISTRE_TRAITEMENTS.md`) : 10 traitements documentés avec finalités, bases légales, durées de conservation, sous-traitants (Supabase, Vercel, Sentry, OVH) — Article 30 RGPD
- □ **Procédure droits des personnes** (`PROCEDURE_DROITS_PERSONNES.md`) : procédure complète pour les 6 droits RGPD (accès, rectification, effacement, portabilité, opposition, limitation) avec délais, vérification d'identité, canaux de réception, motifs de refus — Articles 15-22 RGPD
- □ **Procédure notification CNIL** (`PROCEDURE_NOTIFICATION_CNIL.md`) : gestion d'incident en 5 phases (détection → confinement → évaluation → notification 72h → remédiation), matrice de gravité, registre des violations, contacts d'urgence — Articles 33-34 RGPD
- □ **Procédure revue des droits** (`PROCEDURE_REVUE_DROITS.md`) : revue semestrielle des accès (utilisateurs, RLS, VPS, services tiers), gestion des départs, détection des comptes inactifs, rapport de revue
- □ **Désignation DPO / RSSI** (`DESIGNATION_DPO_RSSI.md`) : Mattias Mathevon désigné DPO et RSSI, missions documentées, organigramme sécurité, plan de formation — Articles 37-39 RGPD
- □ Tous les documents sont dans `documentation/technique/securite/rgpd/`

Version 1.9.0 (3 Avril 2026) — Conformité RGPD

- □ **Export RGPD** (`GET /api/rgpd/export?id_eleve=X`) : export JSON de toutes les données personnelles d'un élève (identité, représentants légaux, formation, paiements, examens, compétences) — Article 20 du RGPD
- □ **Anonymisation RGPD** (`POST /api/rgpd/anonymize`) : remplacement de toutes les PII par [SUPPRIMÉ] (stratégie UPDATE, jamais DELETE — rétention comptable 10 ans) — Article 17 du RGPD
- □ **Confirmation anti-accident** : le mot "ANONYMISER" doit être saisi textuellement pour confirmer l'action irréversible
- □ **Recherche RGPD** (`GET /api/rgpd/search?q=X`) : recherche d'élèves par nom/prénom/email/NEPH, cross-école pour autosoft
- □ **Politique de confidentialité** (`/politique-confidentialite`) : page publique accessible depuis le footer, conforme RGPD (données collectées, finalités, base légale, droits, durées de conservation, sécurité, CNIL)
- □ **Interface admin RGPD** (`/autosoft/rgpd`) : page dédiée dans le panel superadmin avec recherche, export et anonymisation — volontairement hors espace directeur pour éviter les clics accidentels
- □ **Audit trail** : chaque export et anonymisation est tracé dans `audit_log` avec les tables concernées et les erreurs éventuelles
- □ **Accès cross-école pour autosoft** : les API RGPD utilisent `supabaseAdmin` quand l'utilisateur est autosoft, sinon RLS standard

- **Middleware** : `/politique-confidentialite` ajoutée aux routes publiques
- **Tracking consentement** : colonnes `consent_rgpd`, `consent_timestamp`, `consent_ip`, `consent_version` ajoutées à `contrats_eleves` — enregistré à la création et à la signature du contrat

Version 1.8.0 (3 Avril 2026) — Table audit_log

- **Table audit_log** : Migration appliquée en production — `id_ecole`, `id_utilisateur`, `action`, `resource_type`, `resource_id`, `details` (JSONB), `ip_address`, `user_agent`, `created_at`
- **RLS** : Lecture filtrée par école via `get_user_ecole()`, aucune policy INSERT (écriture service role uniquement)
- **Index** : `id_ecole`, `id_utilisateur`, `action`, `created_at` DESC — requêtes rapides même avec beaucoup de logs
- **writeAuditLog()** (`src/lib/security/audit-log.ts`) : utilitaire générique non-bloquant — un échec d'audit ne bloque jamais le flux utilisateur
- **auditLogin()** / **auditLoginFailed()** : raccourcis intégrés dans `/api/auth/login` — chaque connexion (réussie ou échouée) est tracée avec IP et user-agent
- **Extensible** : `writeAuditLog()` prêt à être branché sur toutes les routes critiques (CRUD élèves, contrats, paiements, ANTS)

Version 1.7.0 (3 Avril 2026) — Monitoring Sentry

- **Sentry intégré** via `@sentry/nextjs` — capture erreurs client, serveur et edge
- **Tracing performances** : `tracesSampleRate`: 0.1 en prod (10% — préserve le quota gratuit), 1 en dev
- **Logs applicatifs** : `enableLogs`: true — tous les `console.error/warn` envoyés à Sentry
- **RGPD** : `sendDefaultPii`: false — aucune donnée personnelle (email, IP) transmise à Sentry
- **Source maps** : upload via `SENTRY_AUTH_TOKEN` (GitHub Secrets + Vercel) — stack traces lisibles en prod
- **global-error.tsx** : Error Boundary Next.js qui reporte les erreurs React non catchées à Sentry
- **Environnements distincts** : `environment`: `process.env.NODE_ENV` — filtre prod/dev dans le dashboard
- **CI** : `SENTRY_AUTH_TOKEN` ajouté dans le step Build pour l'upload des source maps

Version 1.6.0 (3 Avril 2026) — CI/CD, SAST, Rate Limiting & Health Check

Phase 0 — Outillage & Qualité

- **ESLint flat config** (`eslint.config.mjs`): `eslint-plugin-security` (eval, BiDi, injection) + `eslint-plugin-react-hooks` + `typescript-eslint` — analyse statique de sécurité à chaque lint
- **TypeScript strict** : Suppression de `ignoreBuildErrors`: true dans `next.config.ts` — les erreurs TS bloquent désormais le déploiement Vercel
- **CI/CD GitHub Actions** (`.github/workflows/ci.yml`): pipeline complet → `npm audit --audit-level=high + tsc -noEmit + ESLint + Vitest + next build` — s'exécute sur chaque PR et push hors main
- **.npmrc legacy-peer-deps=true** : Résout les conflits de peer deps ESLint v10 / `eslint-plugin-react-hooks` — CI et Vercel buildent proprement
- **tsconfig.json** : Dossier `tests/` exclu du check TypeScript principal (Vitest gère ses propres types)

Phase 1 Groupe A — Sécurité active

- **Rate limiting login** (`src/lib/security/rate-limit.ts`): Fenêtre glissante en mémoire, 10 tentatives max / 15 min par IP — réponse 429 avec header `Retry-After`
- **Health check** (`/api/health`): Endpoint public surveillant l'API Next.js + la connexion Supabase en temps réel — requis pour l'intégration API RdvPermis/Livret Numérique (Ministère de l'Intérieur)

Version 1.5.0 (2 Avril 2026) — Application mobile & Sécurité API renforcée

- ☐ **Authentification dual-mode** : `verifyAuth()` accepte désormais le cookie `HttpOnly` (web) ET le Bearer token (mobile) — un seul point d'entrée pour les deux clients
- ☐ **id_ecole / id_moniteur depuis le JWT** : Toutes les routes API lisent ces identifiants depuis le JWT signé, jamais depuis le body ou les query params (anti-falsification)
- ☐ **Toutes les routes API protégées** : Les routes ANTS (`/api/ants/credentials`, `/api/ants/demandes`, `/api/ants/pj`) et la route chat (`/api/chat`) sont désormais protégées par `verifyAuth()`
- ☐ **Application mobile sécurisée** :
 - Stockage JWT via `expo-secure-store` (Keychain iOS / Keystore Android)
 - Bearer token envoyé dans le header `Authorization` sur chaque requête
 - Suppression complète du token et des données au logout
 - Error Boundary global (pas de stack trace exposée à l'utilisateur)
 - Permissions minimales (INTERNET, BIOMETRIC uniquement)
 - Biométrie configurée (Face ID iOS, Fingerprint Android)
- ☐ **EAS Build** : Configuration production avec builds signés cloud, auto-increment des numéros de build
- ☐ **RLS étendu** : 46 tables protégées (ajout des tables ANTS, forfaits, promotions, contrats, etc.)
- ☐ **Tests automatisés** : 26 tests Vitest couvrant auth, sessions, ANTS et paiements
- ☐ **resolveBureau()** : Fonction de résolution de bureau pour les moniteurs multi-bureaux (fallback JWT → body → table enseignants)

Routes corrigées

Les routes suivantes étaient accessibles sans authentification et ont été sécurisées :

Route	Risque avant correction
POST <code>/api/ants/credentials</code>	Écriture de credentials ANTS par un anonyme
GET <code>/api/ants/credentials</code>	Lecture des identifiants ANTS
POST <code>/api/ants/demandes</code>	Création de demandes ANTS frauduleuses
POST <code>/api/ants/pj</code>	Upload de pièces jointes ANTS
POST <code>/api/chat</code>	Abus de l'API Groq (coût, DoS)

Version 1.4.0 (4 Mars 2026) — Impersonation sécurisée

- ☐ **Impersonation admin/autosoft** : Mécanisme de switch de session pour accéder à l'espace directeur d'une auto-école
- ☐ **Double cookie JWT signé** : Session originale sauvegardée dans `autosoft_original_session` (`HttpOnly`, `HS256`)
- ☐ **Vérification rôle ORIGINAL** : Empêche l'escalade de privilèges par impersonation en chaîne
- ☐ **Bypass RLS conditionnel** : `createClientWithRLS()` retourne `supabaseAdmin` uniquement si cookie original valide + rôle `admin/autosoft`
- ☐ **Changement bureau sans BDD** : En impersonation, `update-bureau` re-signe le cookie sans modifier la base
- ☐ **Logout sécurisé** : Suppression des 2 cookies (`autosoft_session` + `autosoft_original_session`) lors du logout
- ☐ **Validation stricte id_ecole** : Vérification type number + entier positif + existence en BDD
- ☐ **Bandeau ImpersonationBanner** : UI indiquant l'état d'impersonation avec bouton de sortie
- ☐ **UserContext enrichi** : Exposition `isImpersonating` et `originalUser` pour le client
- ☐ **Bouton "Accéder en tant que directeur"** : Sur les cartes auto-écoles de l'espace autosoft
- ☐ **Audit de sécurité complet** : Revue de tous les fichiers modifiés, 1 faille critique corrigée (cookie orphelin au logout)
- ☐ **Logs de traçabilité** : Chaque activation/sortie d'impersonation est loggée côté serveur

Faible corrigée

Le logout ne supprimait pas le cookie `autosoft_original_session`. Si un admin impersonnait puis se déconnectait, le cookie orphelin restait valide 7 jours. Un utilisateur se connectant ensuite sur le même navigateur héritait d'un bypass RLS total via `createClientWithRLS()`. Corrigé en supprimant systématiquement les 2 cookies au logout.

Version 1.3.0 (3 Mars 2026) — Configuration ANTS v8 & Certificat CA

- ☐ **Endpoint ANTS v8** : Migration de l'ancien endpoint CERFA vers le vrai WS Auto-École qualification (`ws-autoecole-qlf.ants.gouv.fr`)
- ☐ **Namespace v8** : Correction du namespace SOAP de v9 vers v8 conformément au WSDL officiel
- ☐ **SOAPAction complètes** : Headers SOAPAction avec URL complète (`https://ws-autoecole.ants.gouv.fr/v8/{operation}`)
- ☐ **Variables d'environnement** : `ANTS_SOAP_URL`, `ANTS_SOAP_ACTION_BASE`, `ANTS_NAMESPACE` externalisées (plus aucune URL en dur dans le code)
- ☐ **Validation au démarrage** : Le proxy refuse de démarrer si les variables d'environnement requises sont manquantes
- ☐ **Certificat CA ANTS** : Installé sur le VPS (`ac_services_applicatifs_qlf_5.pem`, converti DER → PEM)
- ☐ **NODE_EXTRA_CA_CERTS** : Configuré dans `.env` et propagé via `PM2 --update-env`
- ☐ **IPv6 Nginx** : Ajout des directives `listen [::]:443 ssl` et `listen [::]:80` pour supporter les connexions IPv6
- ☐ **IP whiteliste** : IP VPS (91.134.143.38) enregistrée auprès de l'ANTS
- ☐ **Connectivité vérifiée** : Curl WSDL HTTP 200 (78 620 octets) + appel SOAP authentifier retourne une réponse SOAP valide
- ☐ **Documentation VPS dédiée** : Création de `VPS-ANTS-PROXY.md` avec architecture, commandes SSH, gestion multi-écoles, et guide de déploiement

Version 1.2.0 (18 Février 2026) — VPS ANTS Proxy & Chiffrement pgcrypto

- ☐ **VPS OVH déployé** : Serveur dédié pour relayer les appels vers l'API ANTS gouvernementale
- ☐ **Nginx HTTPS** : Reverse proxy avec certificat Let's Encrypt (Certbot auto-renewal)
- ☐ **Headers de sécurité Nginx** : HSTS, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy
- ☐ **Rate limiting double couche** : Nginx (10/s) + Express (60/min) sur les routes `/ants/*`
- ☐ **Helmet** : Headers de sécurité HTTP supplémentaires côté Express
- ☐ **CORS restreint** : Uniquement `auto-soft.fr` et déploiements Vercel autorisés
- ☐ **Clé API 256 bits** : Authentification par header `x-api-key` sur toutes les routes ANTS
- ☐ **Trust proxy** : Express configuré pour identifier les IP réelles derrière Nginx
- ☐ **PM2** : Gestion de processus avec auto-restart et startup au boot
- ☐ **Chiffrement pgcrypto** : Credentials ANTS chiffrés en AES-256 dans Supabase (`pgp_sym_encrypt/decrypt`)
- ☐ **Fonctions RPC SECURITY DEFINER** : `upsert_ants_credentials`, `get_ants_credentials`, `check_ants_credentials_exist`
- ☐ **Mot de passe masqué** : L'API GET ne retourne jamais le mot de passe ANTS en clair
- ☐ **Protection XML Injection** : Échappement XML systématique + validation base64 des PJ
- ☐ **Messages d'erreur sanitisés** : Pas de fuite d'informations internes vers le client
- ☐ **Timeout SOAP** : 30 secondes max par appel vers l'ANTS (`AbortController`)

Version 1.1.0 (12 Février 2026) — Correction faille critique

- ☐ **Cookie signé JWT** : Remplacement du JSON en clair par un JWT signé (`jose`, HS256)
- ☐ **Nouveau fichier** : `src/lib/security/session.ts` (`signSession` / `verifySession`)
- ☐ **Middleware sécurisé** : Vérification signature JWT + suppression cookie forgé

- ☐ **verifyAuth async** : Toutes les API routes (38+) utilisent `await verifyAuth()`
- ☐ **Détection côté client** : `HttpContext` redirige vers `/login` si 401
- ☐ **Variable d'environnement** : `SESSION_SECRET` requis dans `.env.local`
- ☐ **Dépendance** : Ajout de `jwt` pour la signature/vérification JWT

Faible corrigée

Le cookie `autosoft_session` contenait un JSON en clair avec le rôle utilisateur. Un attaquant pouvait modifier `"role": "moniteur"` en `"role": "admin"` pour accéder à toutes les fonctionnalités. Maintenant, toute modification du cookie invalide la signature JWT et provoque une déconnexion immédiate.

Version 1.0.0 (5 Février 2026)

- ☐ Implémentation complète du système d'authentification
- ☐ Middleware de sécurité avec contrôle des permissions
- ☐ Row Level Security sur 37 tables
- ☐ Cookies `HttpOnly` sécurisés
- ☐ Auto-refresh des tokens JWT
- ☐ Gestion du bureau virtuel "Tout"
- ☐ Documentation complète de sécurité

☐ Contact

Pour toute question de sécurité, contactez l'équipe Autosoft.

⚠ **IMPORTANT** : Ne jamais partager ce document avec des personnes non autorisées. Il contient des informations sensibles sur l'architecture de sécurité.

Dernière mise à jour : 2 Avril 2026

Auteur : Équipe Autosoft

Version : 1.5.0